



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

DEPARTMENT OF INTELLIGENT SYSTEMS

**DEMONSTRACE VYUŽITÍ PROSTŘEDKŮ ROS 2 NA
PODVOZKU WAVE ROVER S RASPBERRY PI, KAME-
ROU A LIDAREM**

DEMONSTRATION OF THE USE OF ROS 2 ON THE WAVE ROVER CHASSIS WITH RASPBERRY
PI, CAMERA AND LIDAR

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

FILIP BOTLO

VEDOUCÍ PRÁCE

SUPERVISOR

doc. Ing. VLADIMÍR JANOUŠEK, Ph.D.

BRNO 2026

Zadání bakalářské práce



173252

Ústav: Ústav inteligentních systémů (UITS)
Student: **Botlo Filip**
Program: Informační technologie
Název: **Demonstrace využití prostředků ROS 2 na podvozku WAVE ROVER s Raspberry Pi, kamerou a lidarem**
Kategorie: Vestavěné systémy
Akademický rok: 2025/26

Zadání:

1. Nastudujte problematiku mobilní robotiky a teleprezence. Nastudujte metody tvorby softwaru pro mobilní roboty. Seznamte se s middlewarem ROS2.
2. Seznamte se s hardwarem i softwarem platformy WAVE ROVER a s lidarem D300 LD19. Prověřte použitelnost stávajícího kontroleru motorů a senzorů a zvažte možnosti alternativních řešení, včetně možnosti náhrady stávající motorů za motory s enkodéry. Vyžádá-li si to situace, realizujte případné úpravy.
3. Navrhněte systém řízení robota na platformě WAVE ROVER s lidarem s využitím ROS2. Systém poběží distribuovaně na RPi4 nebo RPi5 na robotovi a na stacionárním PC (resp. RPi) s OS Ubuntu 24.4. Systém umožní teleprezenci a přepínání mezi dálkovým ovládáním a autonomním pohybem s vyhýbáním se překážkám.
4. Navržený systém realizujte tak, aby ho bylo možné považovat za demonstraci možností ROS2, včetně dynamické rekonfigurace řídicího software. Uživatel výsledného díla by měl být schopen na základě dokumentace experimentovat s různými konfiguracemi hardware i software (např. použití alternativního IMU, jiného lidarů, motorů s enkodérem i bez apod.).
5. Realizovaný systém otestujte a diskutujte možnosti dalších rozšíření. Součástí odevzdané práce bude plakát a demonstrační video.

Literatura:

- Dle pokynů vedoucího.

Podrobné závazné pokyny pro vypracování práce viz <https://www.fit.vut.cz/study/theses/>

Vedoucí práce: **Janoušek Vladimír, doc. Ing., Ph.D.**
Vedoucí ústavu: Kočí Radek, Ing., Ph.D.
Datum zadání: 1.11.2025
Termín pro odevzdání: 13.5.2026
Datum schválení: 3.11.2025

Abstrakt

Táto práca sa zaoberá mobilnou robotikou a teleprezenciou. Cieľom je navrhnuť a implementovať systém riadenia robota na platforme WAVE ROVER s využitím Robot Operating System 2. Práca zahŕňa prehľad vývoja softvéru pre mobilné roboty v ROS 2, oboznámenie sa s platformou a overenie použiteľnosti existujúceho hardvéru a jeho senzorov. Súčasťou riešenia je vlastný model robota vo formáte URDF/Xacro a jeho nasadenie vo vizualizácii aj vo fyzikálnej simulácii v Gazebo Sim. Navrhovaný systém umožňuje prepínanie medzi diaľkovým ovládaním a autonómnym pohybom s vyhýbaním sa prekážkam. Medzi rozšírenia patrí sledovanie farebného objektu v obraze z kamery. Riadiaci počítač je Raspberry Pi 5 s Ubuntu 24.04.

Abstract

This thesis deals with mobile robotics and telepresence. The aim is to design and implement a robot control system on the WAVE ROVER platform using Robot Operating System 2. The work includes a survey of software development for mobile robots in ROS 2, familiarization with the platform, and verification of the existing hardware and its sensors. The solution includes a custom robot model in URDF/Xacro format and its deployment in visualization and in physical simulation in Gazebo Sim. The proposed system allows switching between remote control and autonomous motion with obstacle avoidance. Among the extensions is tracking of a coloured object in the camera image. The onboard computer is a Raspberry Pi 5 with Ubuntu 24.04.

Klíčové slová

mobilná robotika, ROS 2, WAVE ROVER, Raspberry Pi 5, LiDAR, kamera, teleprezencia, autonómna navigácia, Gazebo, URDF, simulácia, sledovanie objektu

Keywords

mobile robotics, ROS 2, WAVE ROVER, Raspberry Pi 5, lidar, camera, telepresence, autonomous navigation, Gazebo, URDF, simulation, object tracking

Citácia

BOTLO, Filip. *Demonstrace využití prostředků ROS 2 na podvozku WAVE ROVER s Raspberry Pi, kamerou a lidarem*. Brno, 2026. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce doc. Ing. Vladimír Janoušek, Ph.D.

Demonstrace využití prostředků ROS 2 na podvozku WAVE ROVER s Raspberry Pi, kamerou a lidarem

Prehlásenie

Prehlasujem, že som túto bakalársku prácu vypracoval samostatne pod vedením doc. Ing. Vladimíra Janouška, Ph.D. Uviedol som všetky literárne pramene, publikácie a iné zdroje, z ktorých som čerpal.

.....

Filip Botlo
7. mája 2026

Podakovanie

Ďakujem vedúcemu práce doc. Ing. Vladimírovi Janouškovi, Ph.D., za odborné vedenie, pripomienky a trpezlivosť.

Obsah

1	Úvod	4
2	Hardvérové vybavenie	5
2.1	Raspberry Pi 5	5
2.2	Wave Rover	6
2.3	Motory a pohon podvozku	6
2.4	General Driver for Robots	7
2.5	LiDAR a snímač LD19	11
2.6	Raspberry Pi Camera Module 3	11
2.7	Komunikačné rozhrania	12
3	Robot Operating System 2	14
3.1	Úvod, architektúra a zdroje informácií	14
3.2	DDS, domény a micro-ROS	15
3.3	Uzly, správy a priebežná komunikácia	16
3.4	Služby, akcie a kvalita služieb	18
3.5	Workspace, balíky, build, launch a parametre	19
3.6	Nástroje ladenia, transformácie a vizualizácia	21
3.7	Simulácia robotických systémov	22
4	Implementácia	24
4.1	Hardvérová a systémová príprava	24
4.2	Model robota a simulačná vrstva	24
4.3	Implementácia cez UGV Base General (ESP32/HTTP)	26
4.4	Architektúra hlavnej zostavy Wave Roveru	27
4.5	Manuálne ovládanie, motorový uzol a kamera	28
4.6	Korekcia priameho smeru jazdy bez enkodérov	29
4.7	Autonómne blúdenie (wander)	34
4.8	Mapovanie a jednoduchá bodová navigácia	36
4.9	Sledovanie farebnej lopty	37
4.10	Runtime stack, webové rozhranie a prevádzkové služby	42
4.11	Overenie implementácie a prehľad uzlov	44
5	Záver	48
	Literatúra	49
A	Plagát	52

Zoznam obrázkov

2.1	Špecifikácie Raspberry Pi 5 (prehľad platformy)	5
2.2	Wave Rover	6
2.3	Vľavo DC motor, vpravo PWM	7
2.4	General Driver for Robots	8
2.5	Arduino Nano	9
2.6	Vľavo MEMS akcelerometer, vpravo mechanický model	10
2.7	Princíp LiDAR	11
2.8	Raspberry Pi Camera Module 3	12
2.9	Príklad prenosu dát po I ² C	13
2.10	Časový diagram UART	13
3.1	Zásobník klientskych knižníc ROS 2	15
3.2	Model publisher a subscriber na téme	18
3.3	Strom súradnicových rámov tf2	22
4.1	Model Wave Rovera: vľavo RViz2, vpravo Gazebo Sim	26
4.2	Wave Rover – fyzická zostava	27
4.3	Spätnoväzbová slučka regulácie	30
4.4	Priebeh IMU a korekcia pohonu	31
4.5	Sledované body optického toku	32
4.6	Priebeh korekcie podľa optického toku	33
4.7	Porovnanie odchýlky jazdy	34
4.8	Bloková schéma režimu wander	35
4.9	Stavový automat uzla wander	35
4.10	Laserový sken pri režime wander	36
4.11	Graf uzlov pri manuálnom ovládaní a SLAM	37
4.12	Sledovanie objektu: debug obraz	39
4.13	Rozhodovací diagram režimov sledovania lopty (BallController, metóda update()).	40
4.14	Sledovanie objektu: priebeh rýchlostí	40
4.15	Behaviorálny strom sledovania lopty	41
4.16	Webové rozhranie robota	42
4.17	Graf uzlov pri runtime stacku	43
A.1	Plagát k bakalárskej práci	52

Zoznam skratiek

ACK	Acknowledge (potvrdzovací bit).
API	Application Programming Interface (programové rozhranie aplikácie).
BT	Behavior Tree (behaviorálny strom).
CLI	Command-Line Interface (príkazový riadok).
CPU	Central Processing Unit (centrálne procesorová jednotka).
DC	Direct Current (jednosmerný prúd).
DDS	Data Distribution Service (distribučná middleware vrstva v ROS 2).
ESP32	Mikrokontrolérová platforma od Espressif.
GPIO	General Purpose Input/Output (všeobecné vstupno-výstupné piny).
HDR	High Dynamic Range (vysoký dynamický rozsah).
I ² C	Inter-Integrated Circuit (dvojvodičová sériová zbernica).
IMU	Inertial Measurement Unit (inerciálna meracia jednotka).
JSON	JavaScript Object Notation (textový formát na prenos dát).
LD19	2D lidarový snímač použitý na platforme Wave Rover.
MCU	Microcontroller Unit (mikrokontrolér).
MEMS	Micro-Electro-Mechanical Systems (mikroelektromechanické systémy).
Nav2	Navigation2 (navigačný stack pre ROS 2).
PID	Proportional-Integral-Derivative regulátor (proporcionálno-integračno-derivačný regulátor).
PWM	Pulse Width Modulation (šírkovo-pulzná modulácia).
QoS	Quality of Service (kvalita služby komunikácie).
RCL	ROS Client Library (klientská knižničná vrstva ROS).
RMW	ROS Middleware Interface (rozhranie middleware vrstvy ROS).
ROS1	Robot Operating System 1.
ROS 2	Robot Operating System 2.
RPi	Raspberry Pi.
RViz2	Vizualizačný nástroj pre ROS 2.
RX	Receive (príjem dát).
SDA	Serial Data line (dátová linka I ² C).
SCL	Serial Clock line (hodinová linka I ² C).
SLAM	Simultaneous Localization and Mapping (simultánne mapovanie a lokalizácia).
STM32	Rodina 32-bit mikrokontrolérov STMicroelectronics.
ToF	Time of Flight (meranie vzdialenosti podľa času letu signálu).
TX	Transmit (vysielanie dát).
UART	Universal Asynchronous Receiver-Transmitter (asynchrónne sériové rozhranie).
URDF	Unified Robot Description Format (formát popisu robota).
USB	Universal Serial Bus.
WebRTC	Web Real-Time Communication (prenos obrazu/dát v reálnom čase cez web).
WebSocket	Protokol pre obojsmernú komunikáciu klient-server.
WiFi	IEEE 802.11 bezdrôtová lokálna sieť.
Xacro	XML macro language pre parametrizáciu URDF.
YAML	YAML Ain't Markup Language (konfiguračný textový formát).

Kapitola 1

Úvod

Mobilná robotika dnes umožňuje vytvárať autonómne systémy aj na relatívne dostupnom hardvéri. Na prvý pohľad sa môže zdať, že postaviť robota, ktorý sa pohybuje v priestore, vyhýba sa prekážkam a reaguje na okolie, je len otázkou poskladania niekoľkých komponentov. V praxi však takéto riešenie prináša množstvo problémov, akými sú napríklad nepresný odhad polohy, šum v senzoch, obmedzený výkon zariadenia alebo samotná zložitost' prepojenia všetkých častí do jedného funkčného systému.

Táto práca sa zameriava práve na tieto výzvy a ukazuje, ako je možné navrhnuť a postupne vybudovať robotické riešenie založené na ROS 2. Cieľom je vytvoriť zostavu, ktorá zvládne nielen jednoduché diaľkové ovládanie, ale aj základné autonómne správanie. Zároveň je architektúra navrhnutá tak, aby jednotlivé časti boli oddelené a dali sa samostatne upravovať alebo rozširovať.

Riešenie je implementované na podvozku *Wave Rover* od spoločnosti Waveshare, doplnenom o mikropočítač Raspberry Pi 5 a kameru. Zostava je rozdelená medzi samotného robota a externý počítač s Ubuntu 24.04, čo pomáha lepšie využiť dostupný výkon a jednoduchšie testovať jednotlivé časti.

V priebehu práce vysvetlím, ako robot spracúva dáta zo senzorov, ako sa pohybuje aj bez presného merania kolies a ako sa dokáže vyhnúť prekážkam alebo orientovať v prostredí. Okrem toho sú predstavené aj rozšírenia, ako mapovanie prostredia (SLAM), jednoduchá navigácia, simulácia v prostredí Gazebo či sledovanie farebného objektu pomocou kamery.

Práca je rozdelená do viacerých kapitol. Druhá kapitola predstavuje použitý hardvér, tretia približuje princípy ROS 2 a spôsob, akým spolu jednotlivé časti systému komunikujú. Štvrtá kapitola sa venuje samotnému návrhu a implementácii riešenia a záver obsahuje testovanie, vyhodnotenie výsledkov a diskusiu o možnostiach ďalšieho rozvoja.

Kapitola 2

Hardvérové vybavenie

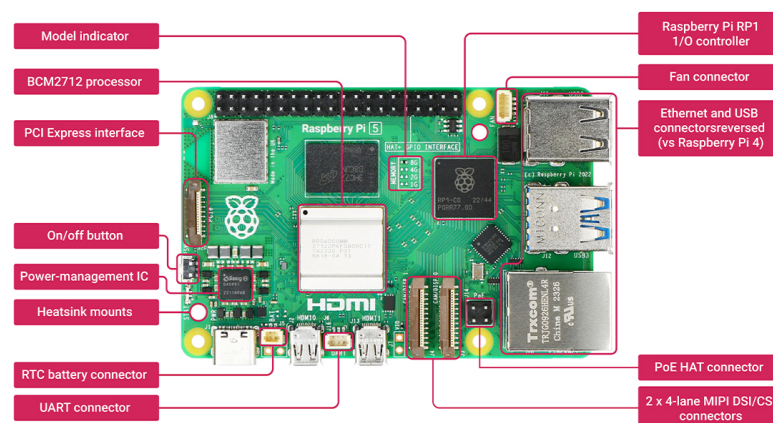
Hardvér robota tvorí základ celého systému a priamo ovplyvňuje jeho možnosti aj obmedzenia. Pri mobilnej platforme nestačí navrhnuť iba softvér, dôležité je poznať aj spôsob riadenia motorov, dostupné senzory, výpočtovú jednotku a rozhrania, cez ktoré spolu jednotlivé časti komunikujú.

Táto kapitola preto predstavuje komponenty použité v práci a zároveň vysvetľuje, akú úlohu majú v navrhnutom riešení. Postupne je opísaný jednodoskový počítač Raspberry Pi, podvozok Wave Rover s pohonom, doplnkové radiace dosky, senzory a komunikačné rozhrania.

2.1 Raspberry Pi 5

Raspberry Pi tvorí hlavnú výpočtovú časť robota. Beží na ňom ROS 2, spracovanie senzorických dát aj väčšina riadiacej logiky. Preto je vhodné najprv stručne priblížiť jeho vlastnosti a úlohu v systéme.

Raspberry Pi 5 je kompaktný jednodoskový počítač určený na vývoj a prototypovanie¹. Často sa používa v projektoch z oblasti elektroniky, internetu vecí a robotiky, pretože ponúka dostatočný výkon pri malých rozmeroch a nízkej spotrebe.



Obr. 2.1: Špecifikácie Raspberry Pi 5 (prehľad platformy)

¹Oficiálna dokumentácia Raspberry Pi: <https://www.raspberrypi.com/documentation/>.

Architektúra

Raspberry Pi 5 je založené na architektúre ARM a využíva procesor Cortex-A76, ktorý má štyri jadrá pracujúce na frekvencii 2,4 GHz. Vďaka tomu dokáže paralelne spracovávať viacero úloh, čo je pri robotickom systéme dôležité najmä pri súčasnom spracovaní dát z kamery, LiDARu, IMU a riadení pohonu.

Na platforme Wave Rover je Raspberry Pi pripojené cez GPIO piny. Tie je možné použiť ako vstupy na čítanie signálov alebo ako výstupy na ovládanie ďalších periférií.

2.2 Wave Rover

Wave Rover predstavuje mobilnú platformu², na ktorej je celé riešenie postavené. Poskytuje podvozok, kolesá, pohon a priestor na rozšírenie o ďalšie komponenty.

Ide o experimentálnu platformu pre mobilnú robotiku: podvozok a kolesá sú navrhnuté tak, aby boli stabilné a odolné voči náročnejším povrhom, v základnej zostave sú senzory a aktuátory a zostáva priestor na modifikácie podľa aplikácie.



Obr. 2.2: Platforma Wave Rover v pôvodnej zostave

2.3 Motory a pohon podvozku

Pohyb robota je zabezpečený jednosmernými motormi. Keďže riadenie pohybu patrí medzi najdôležitejšie časti celej práce, najprv treba vysvetliť, ako takýto motor funguje a ako sa dá jeho rýchlosť a smer ovládať.

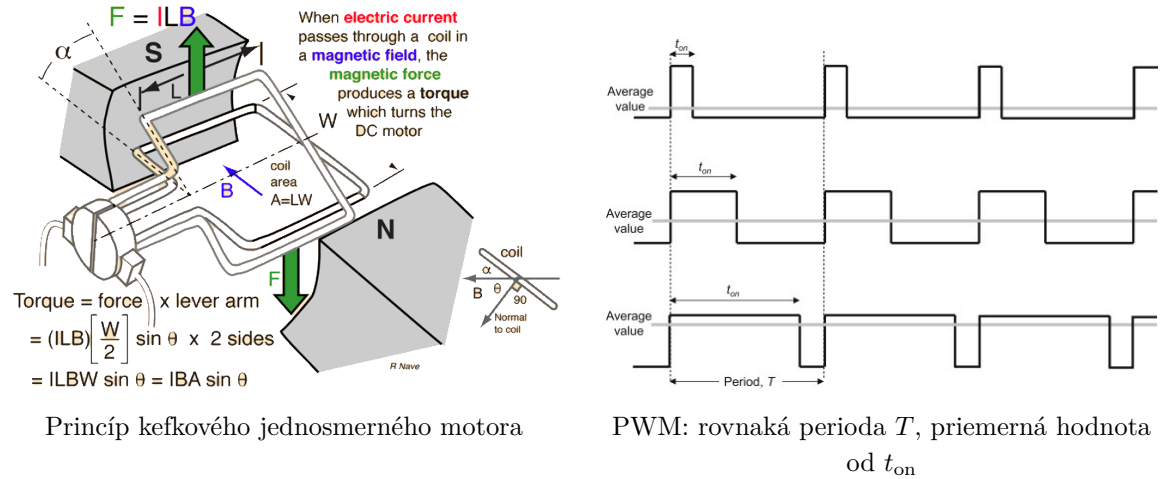
Jednosmerný elektromotor

Pohon kolies zabezpečujú jednosmerné elektromotory. Ich princíp spočíva v elektrickom prúde, ktorý preteká vinutím rotora. Tým v magnetickom poli statora vzniká elektromagnetická sila a následne krútiaci moment na hriadeľ. Aby sa rotor nepootočil iba jednorazovo, ale otáčal sa plynulo, treba priebežne meniť smer prúdu vo vinutí. Motor tak premieňa elektrickú energiu na mechanický pohyb.

²Produktová stránka Wave Rover: <https://www.waveshare.com/wave-rover.htm>.

Rýchlosť otáčania motora sa riadi pomocou PWM (*Pulse Width Modulation*) signálu. PWM nemení priamo veľkosť napätia, ale jeho časový priebeh. Napájanie sa rýchlo prepína medzi zapnutým a vypnutým stavom a pomer medzi týmito stavmi určuje výslednú strednú hodnotu napätia privádzaného na motor. Vďaka tomu je možné regulovať rýchlosť otáčania bez potreby analógovej regulácie napätia.

Zmenu smeru otáčania zabezpečuje H-most, ktorý umožňuje prehodiť polaritu napájania motora [1].



Obr. 2.3: Vľavo ilustrácia princípu kefkového jednosmerného motora. Vpravo šírko-pulzná modulácia (PWM) — pri rovnakej perióde T závisí priemerná hodnota signálu od dĺžky impulzu t_{on} (striedy).

Podvozok robota je diferenciálny. Ľavá a pravá strana pohonu sú riadené nezávisle, čo umožňuje jazdu vpred, vzad aj otáčanie na mieste. V práci sa pracuje s dvomi vetvami riadenia. Hlavná nasadená konfigurácia využíva Motor Driver HAT na Raspberry Pi, kde sa PWM signály pre H-mosty nastavujú cez čip PCA9685 na zbernici I²C [2]. Druhou vetvou je referenčná výrobná alternatíva založená na doske General Driver for Robots s mikrokontrolérom ESP32 a HTTP API od Waveshare, ktorá je opísaná v nasledujúcej časti.

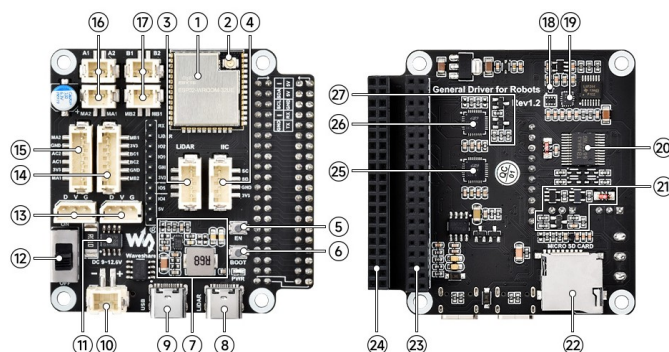
2.4 General Driver for Robots

Doska General Driver for Robots predstavuje výrobnú alternatívu riadenia podvozku. V práci bola použitá najmä na overenie pôvodného spôsobu ovládania platformy a ako porovnanie s vlastnou implementáciou riadenia cez Motor Driver HAT.

General Driver for Robots patrí k ekosystému Waveshare³. Je založený na module ESP32-WROOM-32 a podporuje bezdrôtové rozhrania WiFi, Bluetooth a ESP-NOW. ESP32 tu funguje ako sprostredkovateľ medzi hardvérom a vyššou riadiacou logikou. Obr. 2.4 znázorňuje dosku s číslovaním podľa výrobcu a v tabuľke 2.1 sú uvedené vybrané položky dôležité pre nadväznosť na Raspberry Pi, LiDAR, pohon a palubnú IMU; úplný popis označení uvádza rovnaký zdroj.

³Waveshare Wiki k doske General Driver for Robots: https://www.waveshare.com/wiki/General_Driver_for_Robots.

Pre odhad orientácie doska využíva integrovanú deväťosovú IMU, ktorá pozostáva zo šesťosového senzora pohybu a trojosového magnetometra. Motory ovláda čip TB6612FNG, ďalšie údaje o palubných blokoch, schémach a open-source ukážkach uvádza dokumentácia výrobcu. Prepojenie s ROS 2 cez HTTP a firmvér UGV Base General je podrobnejšie rozobrané v kapitole o implementácii.



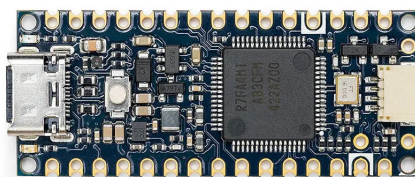
Obr. 2.4: Doska General Driver for Robots (Waveshare) s vyznačenými palubnými zdrojmi; číslovanie zodpovedá tabuľke 2.1 a dokumentácii výrobcu

Č.	Palubný zdroj / rozhranie	Stručný význam pre robotiku
1	ESP32-WROOM-32	Hlavný mikrokontrolér; nízkoúrovňové riadenie a firmvér.
3	Rozhranie LiDAR	Konektor pre LiDAR.
7	Obvod DC–DC 5 V	Napájanie hostiteľa (Raspberry Pi) z palubného napájania.
8–9	Type-C (LiDAR / USB)	Dátové prepojenie LiDARu a USB/UART pre ESP32 (programovanie a sériová linka).
10	Napájací port XH2.54	Vstup DC 7–13 V pre motory a súvisiace okruhy.
14–17	Motor PH2.0 (6P / 2P)	Skupiny A/B: motory s enkodérom (6P) alebo bez enkodéra (2P).
18–19	AK09918C, QMI8658	Palubná deväťosová IMU.
20	TB6612FNG	H-mostové ovládanie jednosmerných motorov.
23–24	40-pinové rozšírenie	Kompatibilný konektor pre osadenie na Raspberry Pi (hostiteľ ROS 2).
25–26	CP2102 (LiDAR, ESP32)	USB–UART mostíky pre dátový tok LiDARu a komunikáciu s ESP32.

Tabuľka 2.1: Výber najdôležitejších palubných blokov dosky General Driver for Robots podľa číslovania (obr. 2.4).

Arduino Nano ako doplnková jednotka pre IMU

Okrem Raspberry Pi a Motor Driver HAT je v realizačnej zostave použité Arduino Nano ako malý mikrokontrolér výhradne na čítanie IMU dát z MPU-6050 a streamovanie údajov po USB. Dôvodom bolo, aby senzor nezdíeľal I²C zbernicu(2.7) s motorovým HATom na strane RPi, čím sa zvýšila spoľahlivosť čítania a zápisu dát pri vyšších frekvenciách pre riadenie motorov aj spracovanie IMU.



Obr. 2.5: Arduino Nano R4 použité v zostave ako mikrokontrolér pre MPU-6050 a USB výstup dát na Raspberry Pi

Inerciálna meracia jednotka (IMU)

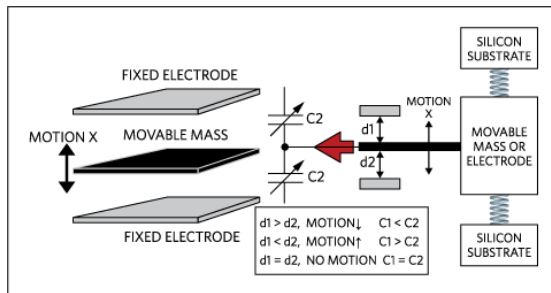
IMU poskytuje informácie o pohybe a orientácii robota. Keďže použitý podvozok nemá enkodéry na kolesách, tieto dáta sú dôležité najmä pri stabilizácii jazdy a odhade orientácie robota.

Akcelerometer

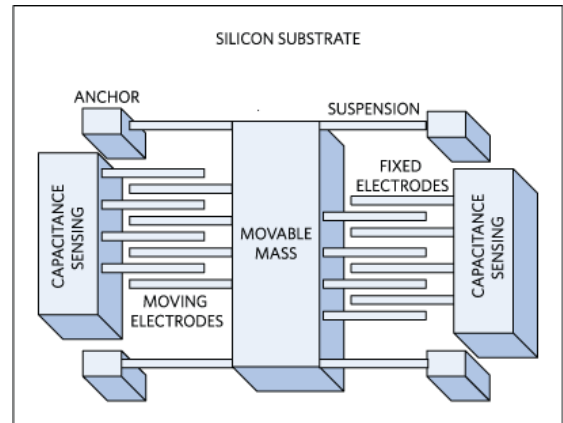
V module IMU MPU-6050 sa nachádza trojosový akcelerometer, ktorý meria lineárne zrýchlenie pôsobiace v osiach senzora. MEMS akcelerometer využíva kapacitný princíp, pri ktorom sa vnútorný hmotný prvok pri zrýchlení vychýľuje. Tým sa mení kapacita štruktúry a táto zmena sa následne prevedie na elektrický signál.

Okrem dynamického zrýchlenia sa v meraní prejavuje aj gravitačné zrýchlenie. Pri pomalom alebo ustálenom pohybe preto možno zo zložiek gravitácie v súradnicovej sústave senzora odhadovať náklon zariadenia voči zemi. V robotických aplikáciách sa tieto údaje využívajú napríklad pri stabilizácii pohybu alebo ako doplnková informácia pre odhad orientácie.

Pri dynamickom pohybe však meranie obsahuje aj zrýchlenie spôsobené samotným pohybom podvozku, čo môže odhad náklonu skresliť. Z tohto dôvodu sa akcelerometer v praxi často kombinuje s gyroskopom alebo s metódami sensorovej fúzie [3], ktoré umožňujú získať stabilnejší odhad orientácie.



Ilustrácia MEMS akcelerometra



Zjednodušený mechanický model: hmotnosť na pružine s tlmením

Obr. 2.6: Akcelerometer meria zrýchlenie cez silové účinky na vnútornú hmotu. Vľavo princíp MEMS realizácie, vpravo ekvivalent hmotnosť–pružina–tlmenie [3].

Gyroskop

Na rozdiel od akcelerometra gyroskop meria uhlovú rýchlosť okolo osí, teda rýchlosť zmeny orientácie. Pri dynamickom pohybe preto často poskytuje presnejší krátkodobý odhad otočenia než samotný akcelerometer. Ten naopak poskytuje dlhodobejšiu referenciu voči gravitácii. Z tejto vlastnosti vychádza aj motivácia kombinovať obe merania v senzorovej fúzii.

Nevýhodou gyroskopu je postupná akumulácia chyby [3]. Pri autonómnych režimoch, v ktorých robot vykonáva otočky, je preto potrebné počítať s obmedzenou presnosťou pri dlhšom behu. Prakticky sa to môže prejavovať aj tým, že odhad orientácie sa pomaly mení, hoci robot stojí na mieste. V digitálnej implementácii zodpovedá integrácii postupné sčítavanie príspevkov z jednotlivých vzoriek, takže každá malá chyba merania sa postupne kumuluje do odchýlky uhla.

Fúzia senzorových dát (komplementárny filter)

Samotný akcelerometer aj gyroskop majú svoje obmedzenia, preto sa ich merania často kombinujú. Prístup vychádza z princípov komplementárnej filtrácie orientácie [4], pričom v implementácii používam jednoduchú diskretnú aproximačnú formu [5].

Z frekvenčného hľadiska sa tento prístup dá chápať ako hornopriepustná vetva pre gyroskop a dolnopriepustná vetva pre akcelerometer. V diskretnom kroku k sa najprv z gyroskopu vypočíta prírastok uhla $\omega_k \Delta t$, kde ω_k je uhlová rýchlosť a Δt perióda vzorkovania. Potom sa predikcia z gyroskopu zmieša s uhlom z akcelerometra v bežne používanej aproximačnej forme:

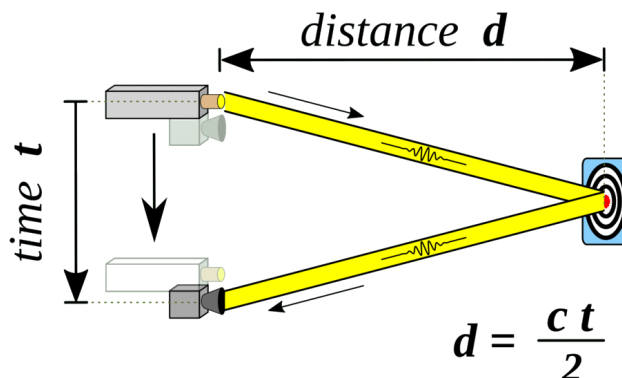
$$\theta_k = \alpha (\theta_{k-1} + \omega_k \Delta t) + (1 - \alpha) \theta_{acc,k}, \quad (2.1)$$

kde $\theta_{acc,k}$ je uhol z akcelerometra a parameter $\alpha \in (0, 1)$ určuje pomer oboch vetiev. Pri vyššom α má väčšiu váhu gyroskop, čo znamená rýchlejší odozvu. Pri nižšom α má väčší vplyv akcelerometer, ktorý viac stabilizuje dlhodobý odhad.

2.5 LiDAR a snímač LD19

Na vnímanie okolia robota je použitý LiDAR senzor. Poskytuje informácie o vzdialenosti prekážok a tieto dáta sú základom pre vyhýbanie sa prekážkam aj pre mapovanie prostredia.

LiDAR (*Light Detection and Ranging*) je optická metóda diaľkového merania [6]. Z viacerých meraní v rôznych smeroch je možné skladať mapy alebo modely okolia. Používa sa napríklad v presnom mapovaní, archeológii, meteorológii aj v navigácii autonómnych vozidiel.



Obr. 2.7: Ilustrácia merania vzdialenosti laserom

Pri rotujúcom LiDARe sa jednotlivé merania vzdialenosti spájajú s uhlom natočenia senzora, čím vzniká dvojrozmerný obraz okolia robota. V ROS 2 sa takýto laserový sken reprezentuje správou `LaserScan`.

Na Wave Roveri je použitý modul *LD19*⁴. Jeho meracie jadro je založené na princípe *direct time of flight*: laserový impulz sa odrazí od cieľa, prijímač zaznamená čas jeho návratu a z časového intervalu letu sa vypočíta vzdialenosť. Údaje sa následne spájajú s uhlom natočenia a odovzdávajú ako sken.

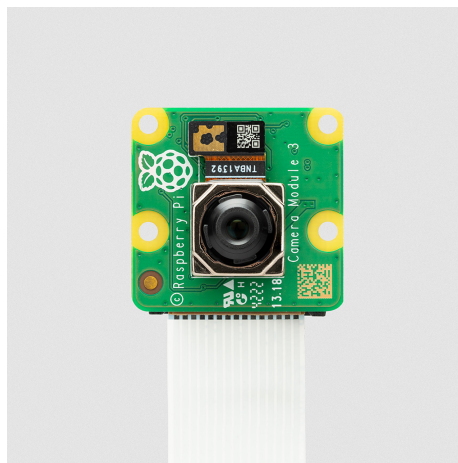
2.6 Raspberry Pi Camera Module 3

Kamera rozširuje schopnosti robota o vizuálne vnímanie prostredia. Okrem teleprezencie je využívaná najmä pri experimentálnych úlohách, ako je korekcia jazdy a sledovanie farebného objektu.

V práci je použitý oficiálny modul Raspberry Pi Camera Module 3⁵. Podrobnejšie technické údaje o rade Camera Module uvádza výrobca v dokumentácii príslušenstva. Snímač využíva Sony IMX708, podporuje HDR snímanie a fázové zaostrovanie, čo uľahčuje ostrenie pri rôznych vzdialenostiach objektov v scéne.

⁴Dokumentácia snímača DTOF LiDAR LD19: https://www.waveshare.com/wiki/DTOF_LIDAR_LD19.

⁵Oficiálna dokumentácia Raspberry Pi Camera Module: <https://www.raspberrypi.com/documentation/accessories/camera.html>.



Obr. 2.8: Raspberry Pi Camera Module 3 (Sony IMX708)

Inštalácia a konfigurácia softvéru (libcamera, balík `camera_ros`, témy obrazu v ROS 2) a väzba na ostatné uzly sú podrobnejšie rozobrané v kapitole 4, sek. 4.5.2.

2.7 Komunikačné rozhrania

Jednotlivé časti robota musia medzi sebou spoľahlivo komunikovať. Používajú sa najmä dve rozhrania: I²C pre komunikáciu s perifériami na zbernici a UART/USB pre sériový prenos dát medzi mikrokontrolérmi, senzormi a Raspberry Pi. V praxi sa v tejto zostave I²C používa najmä pre krátke prepojenia na doske, zatiaľ čo UART/USB prepojuje samostatné zariadenia.

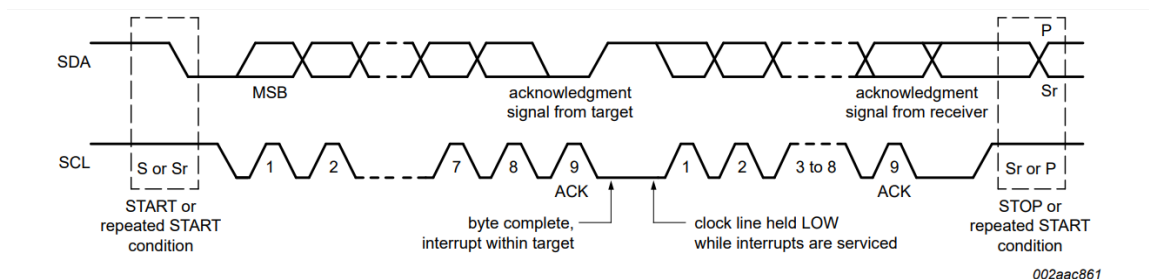
I²C komunikačné rozhranie

I²C (*Inter-Integrated Circuit*) je synchronné sériové komunikačné rozhranie určené na prepojenie integrovaných obvodov a periférnych zariadení na spoločnej zbernici. Komunikácia prebieha po dvoch vodičoch: dátovej linke SDA a hodinovej linke SCL, ktorú generuje riadiace zariadenie (master).

Zbernica umožňuje pripojenie viacerých zariadení paralelne, pričom každé zariadenie má vlastnú adresu. Master začne komunikáciu podmienkou START, následne odošle adresu cieľového zariadenia spolu s bitom určujúcim smer prenosu. Dáta sa prenášajú po bajtoch a po každom bajte nasleduje potvrdzovací bit ACK zo strany prijímateľa. Komunikácia je ukončená podmienkou STOP.

Fyzická realizácia zbernice využíva open-drain výstupy [7]. Zariadenia aktívne ťahajú linky k logickej nule, zatiaľ čo logická jednotka je zabezpečená pomocou pull-up rezistorov pripojených na napájacie napätie. Vďaka tomu môže zbernica zdieľať viac zariadení.

V rámci tejto práce je I²C zbernica využitá najmä na komunikáciu s riadiacim obvodom motorov a inerciálnym senzorom.



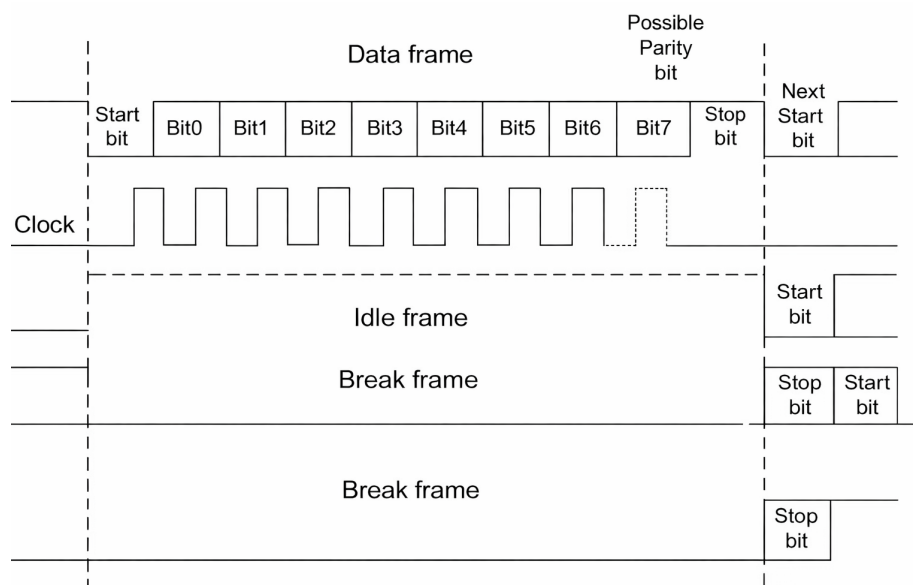
Obr. 2.9: Príklad prenosu dát po I²C

UART a USB

UART (*Universal Asynchronous Receiver-Transmitter*) [8] je asynchrónne sériové rozhranie. Dáta sa prenášajú po jednotlivých bitoch bez zdieľaného hodinového signálu. V mikrokontroléri ide o hardvérovú perifériu, ktorá z paralelných slov procesora vytvára sériový výstup a z prichádzajúceho toku bitov skladá bajty pre CPU.

Obe strany komunikácie musia byť nastavené na rovnakú symbolovú rýchlosť (*baud rate*), ktorá určuje časový úsek jedného bitu. Ak sa rýchlosti nezhodujú, prijímač nesprávne rozdelí rámec a vznikajú chyby. Prenos je najčastejšie dvojlinkový v plnom duplexe: vodič TX jedného zariadenia sa pripája na RX druhého a opačne, pričom obe zariadenia musia mať spoločnú referenčnú zem.

V nečinnom stave je linka v logickej jednotke. Prenos začne štartovým bitom, pri ktorom linka prejde do logickej nuly. Nasledujú dátové bity, prípadne paritný bit a na konci jeden alebo viac stop bitov, ktoré vrátia linku späť do pokojového stavu. Prijímač sa zosynchronizuje podľa prechodu štartového bitu a následne číta jednotlivé bity v pravidelných intervaloch.



Obr. 2.10: Časový diagram UART: rámec so štartovým bitom, dátovými bitmi, voliteľnou paritou a stop bitom.

Kapitola 3

Robot Operating System 2

ROS 2 v navrhnutom riešení slúži ako komunikačná vrstva medzi motorovým uzlom, senzormi, kamerou, webovým rozhraním, simuláciou a autonómnymi modulmi. Aby bolo jasné, ako sú jednotlivé časti prepojené, táto kapitola stručne predstavuje koncepty ROS 2, ktoré sa priamo využívajú pri implementácii robota.

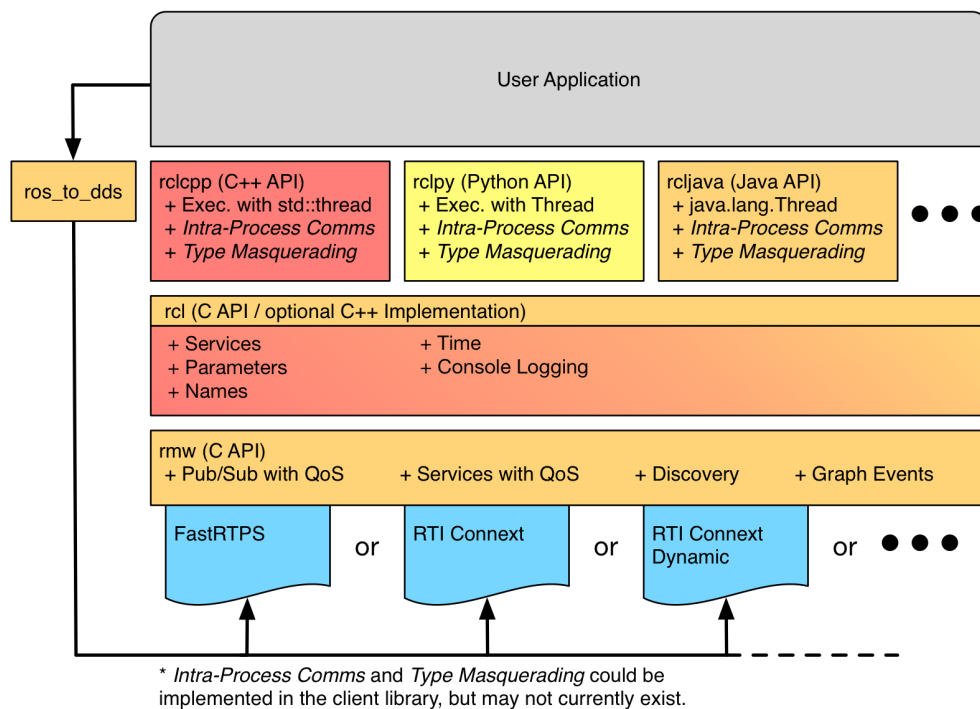
Najprv je opísaná základná architektúra ROS 2 a komunikácia cez DDS. Ďalej sú vysvetlené uzly, témy, služby, akcie a parametre, teda prvky, z ktorých je zložená výsledná zostava. Záver kapitoly sa venuje nástrojom na ladenie, transformáciám a simulácii, ktoré boli použité pri vývoji a overovaní riešenia.

3.1 Úvod, architektúra a zdroje informácií

ROS 2 je middleware platforma určená na vývoj robotických aplikácií. Ide o sadu knižníc, nástrojov a rozhraní, ktoré zjednocujú komunikáciu medzi komponentmi robotického systému. ROS 2 poskytuje štandardizovaný spôsob práce s uzlami, správami, témami, službami, akciami, parametrami a transformáciami, čím uľahčuje návrh modulárnych a distribuovaných aplikácií.

V porovnaní s ROS1 je navrhnutý na robustnejšie nasadenie v distribuovaných systémoch, podporu viacerých platforiem a flexibilnejšiu komunikáciu cez DDS. Architektúra oddeľuje aplikačný kód od transportnej vrstvy a umožňuje meniť middleware bez zásahu do logiky uzlov.

Väčšina tejto kapitoly vychádza z oficiálnej dokumentácie ROS 2 [9], takže priebežné vysvetlenia a príklady budú spracované na základe tohto zdroja.



Obr. 3.1: Zásobník klientskych knižníc a rozhraní: aplikácia cez `rclpy/rclcpp`, spoločná vrstva RCL, RMW a DDS ako transport

3.2 DDS, domény a micro-ROS

V tomto riešení beží zostava distribuovane medzi Raspberry Pi a externým počítačom. V krátkosti vysvetlím, ako ROS 2 zabezpečuje komunikáciu medzi uzlami v sieti a ako sa jednotlivé behy oddeľujú pomocou domén.

DDS a discovery

Komunikačný základ ROS 2 tvorí DDS [10], čo je decentralizovaný publish–subscribe middleware. DDS v praxi zabezpečuje priamu komunikáciu medzi uzlami bez centrálného servera. Po spustení uzla dochádza k jeho automatickému oznámeniu v rámci rovnakej domény. Následne sa vytvorí spojenie s kompatibilnými účastníkmi komunikácie.

Výber, ktorá logická DDS doména sa použije, sa riadi premennou prostredia `ROS_DOMAIN_ID`. Uzly so zhodným ID patria do jednej skupiny pre discovery a výmenu správ, odlišná hodnota oddelí napríklad viac nezávislých behov ROS 2 na jednom počítači alebo susedné robotické systémy v lokálnej sieti.

micro-ROS a embedded systémy

Pre mikrokontroléry s obmedzenou pamäťou a výkonom nie je plný ROS 2 stack na zariadení reálny a preto v ekosystéme existuje *micro-ROS*¹. Ide o zjednodušené klientské knižnice, ktoré na MCU zachovávajú základné koncepty ROS 2 (uzly, témy, služby, para-

¹Oficiálna dokumentácia micro-ROS: <https://micro.ros.org/>.

metre). Typicky beží *micro-ROS klient* na zariadení ako ESP32 či STM32 a *micro-ROS agent* na výkonnejšom uzle zabezpečí premostenie do tej istej DDS domény ako ostatné uzly.

3.3 Uzly, správy a priebežná komunikácia

Výsledný systém je rozdelený na viacero samostatných uzlov, z ktorých každý rieši konkrétnu úlohu, napríklad čítanie senzora, riadenie motorov alebo spracovanie obrazu. Ďalej ukazujem tento komunikačný model ROS 2.

Uzly a klientské knižnice

Základnou jednotkou aplikácie je uzol (*node*). Uzol, ako som hovoril v úvode, predstavuje samostatný proces alebo logickú časť procesu, ktorá vykonáva určitú funkcionálnu.

Nasledujúca ukážka ilustruje najjednoduchšiu kostru uzla v `rclpy`:

```
1      import rclpy
2      from rclpy.node import Node
3
4      class SimpleNode(Node):
5          def __init__(self):
6              super().__init__('simple_node')
7              self.get_logger().info('Uzol bol spustený')
8
9      def main(args=None):
10         rclpy.init(args=args)
11         node = SimpleNode()
12         rclpy.spin(node)
13         node.destroy_node()
14         rclpy.shutdown()
```

Správy a rozhrania

Komunikácia medzi uzlami prebieha pomocou správ (*messages*), ktorých formát je pevne definovaný rozhraním. Štandardné typy správ sú súčasťou balíkov ako `std_msgs`, `geometry_msgs`, `sensor_msgs` alebo `nav_msgs` (definície typov sú súčasťou ekosystému ROS 2).

Pre túto prácu sú dôležité napríklad typy:

- `sensor_msgs/msg/LaserScan` pre 2D lidar,
- `sensor_msgs/msg/Image` pre kameru,
- `sensor_msgs/msg/Imu` pre IMU,
- `geometry_msgs/msg/Twist` pre rýchlostné príkazy,
- `nav_msgs/msg/Odometry` pre odometriu.

Témy a model publisher–subscriber

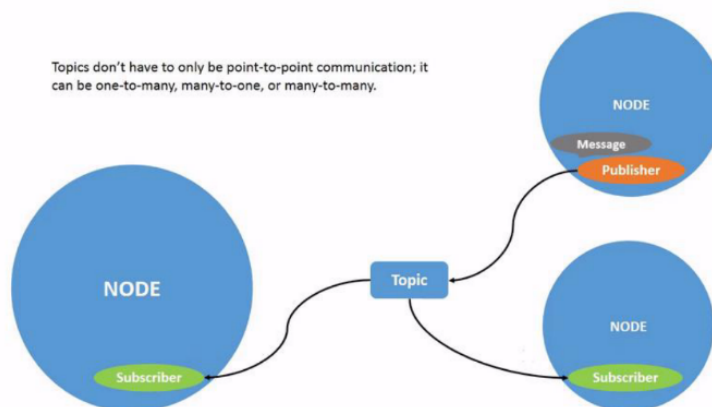
Témy (*topics*) sú asynchrónne komunikačné kanály určené na priebežnú výmenu správ medzi uzlami. Publisher odosiela správy na tému a subscriber ich prijíma. Tento model je vhodný pre kontinuálne dátové toky zo senzorov alebo z riadiacich uzlov.

Príklad jednoduchého publishera v jazyku Python:

```
1      from std_msgs.msg import String
2      import rclpy
3      from rclpy.node import Node
4
5      class Talker(Node):
6          def __init__(self):
7              super().__init__('talker')
8              self.pub = self.create_publisher(String, 'demo_topic', 10)
9              self.timer = self.create_timer(0.5, self.timer_callback)
10             self.counter = 0
11
12         def timer_callback(self):
13             msg = String()
14             msg.data = f'Sprava {self.counter}'
15             self.pub.publish(msg)
16             self.counter += 1
```

Zodpovedajúci subscriber môže mať tvar:

```
1      from std_msgs.msg import String
2      import rclpy
3      from rclpy.node import Node
4
5      class Listener(Node):
6          def __init__(self):
7              super().__init__('listener')
8              self.sub = self.create_subscription(
9                  String, 'demo_topic', self.callback, 10)
10
11         def callback(self, msg):
12             self.get_logger().info(f'Prijata sprava: {msg.data}')
```



Obr. 3.2: Publisher publikuje správy na tému a jeden alebo viac subscriberov ich odoberá

3.4 Služby, akcie a kvalita služieb

Nie všetka komunikácia má rovnaký charakter. Senzorické dáta sa posielajú priebežne, prepnutie režimu je jednorazová požiadavka a sledovanie lopty je dlhšie trvajúca úloha. Preto ROS 2 rozlišuje témy, služby a akcie.

Služby

Služby (*services*) realizujú synchronný model request–response. Klient odošle požiadavku a server vráti odpoveď. Tento spôsob komunikácie sa používa na jednorazové operácie, ktoré majú mať jednoznačný výsledok, ako je reset modulu, prepnutie režimu alebo získanie konfigurácie.

Zjednodušený príklad servera služby:

```

1  from example_interfaces.srv import AddTwoInts
2  import rclpy
3  from rclpy.node import Node
4
5  class AddService(Node):
6      def __init__(self):
7          super().__init__('add_service')
8          self.srv = self.create_service(
9              AddTwoInts, 'add_two_ints', self.callback)
10
11      def callback(self, request, response):
12          response.sum = request.a + request.b
13          return response

```

Akcie

Akcie (*actions*) sú určené pre dlhšie trvajúce operácie. Na rozdiel od služieb umožňujú priebežnú spätnú väzbu, vrátenie konečného výsledku a zrušenie požiadavky. Využívajú napríklad pri navigácii, manipulácii alebo vykonávaní komplexnej sekvencie pohybov.

Typická akcia zahŕňa:

- odoslanie cieľa (*goal*),
- priebežný feedback,
- finálny výsledok (*result*).

Zjednodušená myšlienka callbacku akčného servera:

```
1     def execute_callback(self, goal_handle):
2         sequence = [0, 1]
3         for i in range(2, goal_handle.request.order):
4             sequence.append(sequence[i-1] + sequence[i-2])
5             goal_handle.publish_feedback(...)
6         goal_handle.succeed()
7         result = Fibonacci.Result()
8         result.sequence = sequence
9         return result
```

QoS

Dôležitou vlastnosťou ROS 2 je podpora Quality of Service (QoS), ktorá umožňuje ovplyvniť správanie komunikácie medzi uzlami. Pomocou QoS je možné nastaviť, či je pri prenose dát dôležitejšia spoľahlivosť (aby sa nestratili správy), alebo nízka latencia (aby boli dáta čo najaktuálnejšie).

Medzi najdôležitejšie QoS politiky patria:

- **history** a **depth** – určujú, koľko posledných správ sa uchováva v pamäti,
- **reliability** – definuje, či má byť prenos spoľahlivý (bez straty dát) alebo najlepšou snahou (môžu sa stratiť niektoré správy),
- **durability** – určuje, či noví odberatelia dostanú aj staršie správy,
- **deadline** – maximálny čas medzi správami,
- **lifespan** – ako dlho je správa platná,
- **liveliness** – mechanizmus na kontrolu, či je uzol stále aktívny.

Voľba QoS závisí od konkrétneho použitia. Zo skúsenosti sa osvedčilo, že pri senzorických dátach, ako sú dáta z LiDARu alebo kamery, je lepšie uprednostniť aktuálnosť než úplnú spoľahlivosť a vybrať režim *best effort*. Pri riadiacich príkazoch je zasa vhodnejšie použiť spoľahlivý prenos.

3.5 Workspace, balíky, build, launch a parametre

Keďže systém pozostáva z viacerých balíkov a uzlov, bolo potrebné riešiť aj ich organizáciu, spúšťanie a konfiguráciu. Na to slúži ROS 2 workspace, launch súbory a parametre.

Workspace, balíky a build systém

Vývoj v ROS 2 prebieha vo workspace, ktorého zdrojová časť sa spravidla nachádza v priečinku `src/`. Po príkaze `colcon build` vznikajú priečinky `build/`, `install/` a `log/`. Dokumentácia Jazzy rozlišuje aj vzťah *underlay* a *overlay*, kde systémová inštalácia ROS 2 tvorí základ a vlastný workspace na ňu nadväzuje.

Príklad typickej štruktúry Python balíka:

```
1      my_package/  
2      |-- package.xml  
3      |-- setup.py  
4      |-- setup.cfg  
5      |-- resource/  
6      |-- my_package/  
7      |   |-- __init__.py  
8      |   '-- my_node.py  
9      |-- launch/  
10     '-- config/
```

Launch súbory

Launch súbory slúžia na spustenie viacerých uzlov, nastavenie parametrov, remapovanie tém a celkové zostavenie aplikácie. V ROS 2 sa bežne používajú Python launch súbory, ktoré vracajú objekt `LaunchDescription`.

Jednoduchý launch súbor môže vyzeráť nasledovne:

```
1      from launch import LaunchDescription  
2      from launch_ros.actions import Node  
3  
4      def generate_launch_description():  
5          return LaunchDescription([  
6              Node(  
7                  package='demo_nodes_py',  
8                  executable='talker',  
9                  name='demo_talker'  
10             )  
11          ])
```

Parametre

Parametre sú konfiguračné hodnoty viazané na konkrétny uzol. Možno ich nastavovať pri štarte aj za behu a ich životnosť sa viaže na životnosť uzla.

Príklad deklarácie a čítania parametra v `rclpy`:

```
1      import rclpy  
2      from rclpy.node import Node  
3  
4      class ParamNode(Node):  
5          def __init__(self):  
6              super().__init__('param_node')  
7              self.declare_parameter('gain', 0.5)
```

```

8         gain = self.get_parameter('gain').value
9         self.get_logger().info(f'gain = {gain}')

```

Parametre sa zvyknú ukladať do YAML súborov a predávajú cez launch súbory, čo zjednodušuje konfiguráciu viacerých uzlov bez zásahu do zdrojového kódu.

3.6 Nástroje ladenia, transformácie a vizualizácia

Pri vývoji robota nestačilo uzly iba spustiť, bolo potrebné aj sledovať, či medzi sebou správne komunikujú a či dáta zo senzorov dávajú zmysel. Preto boli pri práci užitočné nástroje na introspekciu, vizualizáciu a ladenie.

Príkazový riadok a introspekcia systému

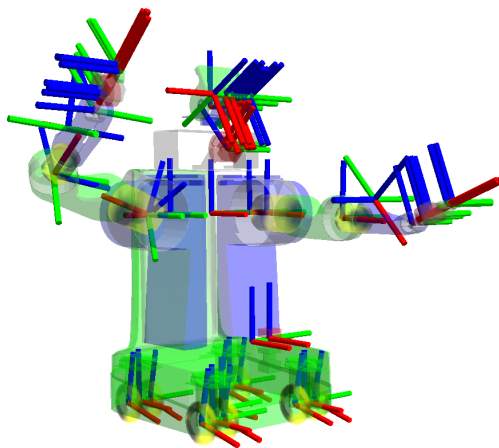
ROS 2 obsahuje rozsiahle CLI nástroje na introspekciu systému, ktoré umožňujú sledovať stav uzlov, komunikáciu a konfiguráciu bez zásahu do kódu. Medzi najdôležitejšie patria:

- **ros2 node list** – zobrazí zoznam všetkých aktívnych uzlov v systéme, **ros2 node info** poskytne detailné informácie o konkrétnom uzle (publikované a odoberané témy, služby),
- **ros2 topic list** – vypíše dostupné témy, **ros2 topic echo** umožňuje sledovať obsah správ publikovaných na téme, **ros2 topic hz** odhaduje frekvenciu publikovania,
- **ros2 service list** – zobrazí dostupné služby, **ros2 service call** umožňuje manuálne zavolať službu s konkrétnymi parametrami,
- **ros2 param list** – vypíše parametre uzlov, **ros2 param get** a **ros2 param set** slúžia na čítanie a zmenu hodnôt parametrov počas behu,
- **ros2 interface show** – zobrazí definíciu typu správy, služby alebo akcie (napr. štruktúru `sensor_msgs/msg/LaserScan`),
- **ros2 doctor** – diagnostický nástroj, ktorý kontroluje správnosť konfigurácie ROS 2 prostredia a upozorňuje na možné problémy.

tf2 a súradnicové rámce

Subsystem tf2 slúži na reprezentáciu vzťahov medzi súradnicovými rámami robota. Každá transformácia má rodičovský a detský rám, čím vzniká strom transformácií. Bežne sa používajú rámce ako `map`, `odom`, `base_link` alebo rámce jednotlivých senzorov. Robot to potrebuje na správne prepočty polohy medzi mapou, podvozkom a senzormi.

Statické transformácie opisujú pevné geometrické vzťahy, napríklad montáž senzora na podvozok. Dynamické transformácie sa menia v čase, čo je napríklad poloha robota voči odometrii. V `rclpy` sa na publikovanie transformácií používa `TransformBroadcaster` a na ich odoberanie kombinácia `Buffer` a `TransformListener`.



Obr. 3.3: Príklad stromu súradnicových rámov prepojených transformáciami tf2, rodičovské a detské rámce zodpovedajú vzťahom medzi mapou, odometriou, podvozkom a senzormi

RViz2 a rqt

RViz2 je hlavný vizualizačný nástroj v prostredí ROS 2, ktorý umožňuje zobrazenie stavu robota a dát zo senzorov v reálnom čase. Podporuje vizualizáciu 3D modelu robota, laserových skenov, obrazových dát, markerov, máp, plánovaných trajektórií a transformácií medzi súradnicovými rámami. Dá sa v ňom dynamicky pridávať jednotlivé zobrazovacie prvky a konfigurovať ich parametre podľa potreby. V rámci tejto práce sa využíva RViz na výkonnejšom externom počítači 4.8.

Balík rqt združuje sadu grafických nástrojov, ktoré slúžia na analýzu a ladenie systému. Medzi najčastejšie používané patria:

- **rqt_graph** – zobrazuje graf uzlov a tém, čo umožňuje overiť štruktúru komunikácie a prepojenie jednotlivých komponentov,
- **rqt_plot** – umožňuje sledovať časový priebeh číselných veličín publikovaných na témach, napríklad výstupy regulátora alebo hodnoty zo senzorov,
- **rqt_image_view** – slúži na zobrazenie obrazových dát zo senzorov, napríklad kamery alebo spracovaného obrazu.

3.7 Simulácia robotických systémov

Simuláciu som využil na overenie modelu robota, senzorov a správania ešte pred nasadením na fyzický hardvér. Detailnejšie nastavenie a konkrétne použitie uvádzam v kapitole o implementácii.

Gazebo a simulácia

ROS 2 podporuje simuláciu robotických systémov prostredníctvom nástrojov, ako je Gazebo Sim [11]. Pred samotnou prácou s hardvérom som v simulácii testoval algoritmy aj so simulovanými senzormi a fyzikou prostredia.

Prepojenie medzi ROS 2 a Gazebo Sim sa rieši pomocou balíkov **ros_gz_sim** a **ros_gz_bridge**, ktoré zabezpečujú komunikáciu medzi simulačným prostredím a ROS témami.

Konkrétnu konfiguráciu a spôsob využitia týchto nástrojov podrobnejšie opisujem v kapitole 4, najmä v sekcii 4.2.

Kapitola 4

Implementácia

V tejto kapitole sa venujem návrhu a realizácii výsledného systému. Postupne prejdem prípravu vývojového prostredia, modelovanie robota so simulačnou vrstvou, konfiguráciu s pôvodným driverom a nakoniec hlavnú implementáciu.

Neopisujem len samotné implementačné kroky, ale aj to, ako sú jednotlivé časti navrhnuté tak, aby spolu fungovali ako modulárny a rozšíriteľný systém. Dôležitým aspektom je rozdelenie funkcionality do samostatných uzlov a ich komunikácia cez ROS 2.

Pri vypracovaní tejto bakalárskej práce som sa čiastočne inšpiroval bakalárskou prácou Daniela Onderku [12] z hľadiska štruktúry textu a taktiež pri niektorých implementačných prvkoch: režim blúdenia (wander), jednoduchá bodová navigácia a Gazebo simulácia.

4.1 Hardvérová a systémová príprava

Ešte pred samotným spozajznením robota bolo potrebné pripraviť jednotné prostredie na Raspberry Pi. Ako základ softvérového stacku je operačný systém Ubuntu 24.04 a k nemu distribúcia ROS 2 Jazzy. Súčasťou prípravy bolo aj nastavenie `ROS_DOMAIN_ID`, ktoré zabezpečuje, aby uzly bežiacie na rôznych zariadeniach patrili do rovnakej komunikačnej domény a mohli si medzi sebou vymieňať správy.

Osobitnú pozornosť si vyžadovala integrácia kamery na Raspberry Pi 5. Podrobný opis problémov s `/dev/video*`, vrstvou *libcamera* a voľbou balíka `camera_ros`¹ je uvedený v sekcii 4.5.2. V tejto časti chcem len spomenúť, že pre stabilnú prevádzku sa osvedčil práve tento prístup.

4.2 Model robota a simulačná vrstva

Súčasťou návrhu systému bolo vytvorenie jednotného modelu robota. Vytvorenie modelu robota je dôležité pre simulácie, vďaka čomu sa dajú testovať algoritmy ešte skôr, ako spustím reálneho robota.

Model je vytvorený vo formáte Xacro a rozdelený na viacero logických častí. Základ tvorí telo robota reprezentované rámom `base_link`, ku ktorému sú pripojené kolesá a jednotlivé senzorické bloky. Pri definovaní modelu nebola doplnená len vizuálna reprezentácia, ale aj

¹Praktický postup pre RPi5 a ROS 2 Jazzy: <https://github.com/ARLunan/Raspberry-Pi-Camera-ROS/blob/main/Documents/RPCamera-Installation-RP5.md>. Oficiálny repozitár `camera_ros`: https://github.com/christianrauch/camera_ros.

kolízne a zotrvačné parametre, aby model neslúžil iba na zobrazenie, ale aj na fyzikálnu simuláciu v Gazebo.

4.2.1 Definícia tela

Podvozok som definoval ako jednoduchý kváder s príslušnou hmotnosťou, geometriou a aproximačnou maticou zotrvačnosti:

```
1      <link name="base_link">
2          <inertial>
3              <mass value="1.2"/>
4              <origin xyz="0 0 0"/>
5              <inertia ixx="0.003" iyy="0.005" izz="0.006"/>
6          </inertial>
7          <visual>
8              <origin xyz="0 0 0"/>
9              <geometry><box size="{body_l} {body_w} {body_h}"/></geometry>
10             <material name="dark_gray"/>
11          </visual>
12          <collision>
13              <origin xyz="0 0 0"/>
14              <geometry><box size="{body_l} {body_w} {body_h}"/></geometry>
15          </collision>
16      </link>
```

Hodnoty zotrvačnosti v ukážke vyššie nezodpovedajú presnému rozloženiu na reálnom podvozku, no presné hodnoty by si vyžadovali detailnejšie meranie rozloženia hmotnosti, čo presahuje rámec tejto práce. Aktuálne hodnoty teda postačia.

Kolesá sú k telu podvozku pripojené rotačnými kĺbmi. V simulačnej vrstve boli doplnené aj základné parametre kontaktu a trenia, aby sa správanie modelu aspoň približne podobalo fyzickému podvozku.

4.2.2 Kamera a LiDAR

Na ten istý základný rám boli pomocou pevných kĺbov pripojené aj kamera a LiDAR.

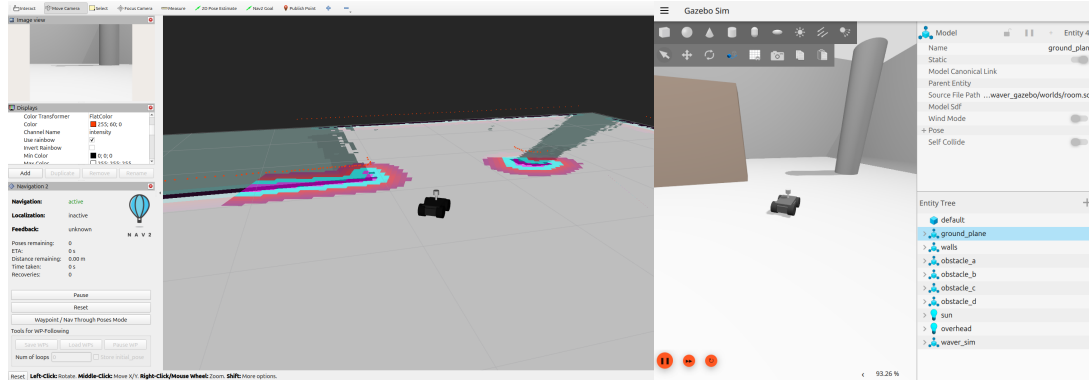
```
1      <joint name="camera_joint" type="fixed">
2          <parent link="base_link"/>
3          <child link="camera_link"/>
4          <origin xyz="0.097 0 0.005" rpy="0 0 0"/>
5      </joint>
6      <joint name="lidar_joint" type="fixed">
7          <parent link="base_link"/>
8          <child link="lidar_link"/>
9          <origin xyz="-0.04 0 0.092" rpy="0 0 0"/>
10     </joint>
```

4.2.3 Gazebo Sim a RViz2

Takto pripravený model bol následne použitý v RViz2 aj v Gazebo Sim [11] – vizualizácia aj fyzikálna simulácia vychádzajú z tej istej štruktúry. V simulačnom prostredí sú kamera

aj LiDAR sprístupnené cez `ros_gz_bridge`, vďaka čomu s nimi viem pracovať ako so štandardnými ROS témami, napríklad `/camera/image_raw` a `/scan`.

Simulácia mi umožnila prvotné vyskúšanie manuálneho aj autonómneho riadenia robota, vrátane experimentov s navigáciou v štýle Nav2 v simulácii. Oproti reálnemu prostrediu však pretrvávajú rozdiely v trení, poddajnosti kolies, oneskorení senzorov a šume.



Obr. 4.1: Model Wave Rovera: vľavo RViz2, vpravo Gazebo Sim

4.3 Implementácia cez UGV Base General (ESP32/HTTP)

Ešte pred vytvorením hlavnej implementácie bola overená aj referenčná konfigurácia založená na doske General Driver For Robots. Táto konfigurácia bola skúmaná ako alternatíva k priamemu riadeniu motorov z Raspberry Pi, s cieľom overiť flexibilitu a možnosti existujúceho firmvéru platformy.

Riadenie v tejto konfigurácii vychádza z oficiálneho firmvéru UGV Base General². Architektúra je rozdelená medzi dve samostatné výpočtové jednotky. Raspberry Pi plní úlohu nadradeného počítača, na ktorom beží ROS 2 a vyššie vrstvy riadenia, zatiaľ čo ESP32 zabezpečuje nízkoúrovňové ovládanie motorov a priamu komunikáciu s hardvérom podvozku.

Komunikácia medzi oboma zariadeniami prebieha prostredníctvom HTTP požiadaviek s JSON obsahom. Raspberry Pi vďaka tomuto prístupu nemuselo byť fyzicky pripojené priamo k motorovému radiču ani pevne umiestnené na podvozku robota. Aj keď takýto spôsob integrácie nie je v prostredí ROS 2 taký bežný, prináša v tomto prípade väčšiu flexibilitu a jednoduchosť.

Na rozdiel od natívnej komunikácie v ROS 2, HTTP API firmvéru je striktne *request-response*. Každý príkaz vyžaduje novú požiadavku a čakanie na odpoveď. Pre časovo kritické uzatvorené riadenie s vysokou frekvenciou a nízkym jitterom to nie je ideálne, lebo latencia závisí od zaťaženia ESP32, WiFi a parsovania JSON. Napriek tomu bol tento prístup vhodný na rýchle prototypovanie a overenie pohonu bez úprav výrobného firmvéru.

4.3.1 HTTP komunikácia s ESP32

Všeobecný tvar komunikácie je:

1 `http://<ESP32_IP>/js?json=<URL_ENCODED_JSON>`

²Oficiálny repozitár firmvéru: https://github.com/waveshareteam/ugv_base_general.

Každá správa obsahuje pole T, ktoré určuje typ operácie, a ďalšie parametre podľa požadovanej funkcie. Pri riadení pohybu sa používajú hodnoty ľavej a pravej strany pohonu. Príkladom je príkaz {"T":1,"L":0.3,"R":0.3}, ktorý nastaví obe strany na rovnakú doprednú rýchlosť. Hodnoty L a R sú normalizované v rozsahu od -1 do 1.

V prostredí ROS 2 túto komunikáciu zabezpečujú jednoduché mostové uzly, ktorých úlohou je preberať vstupy z tém alebo z klávesnice a prevádzať ich na HTTP požiadavky smerované na ESP32. Ide o preklad medzi internou komunikáciou ROS 2 a externým API. Každá zmena príkazu znamená aspoň jeden sieťový round-trip navyše oproti priamemu volaniu ovládača na I²C z Raspberry Pi.

Aj z tohto dôvodu bola táto architektúra použitá len na experimentálne účely a hlavná implementácia bola realizovaná priamo na Raspberry Pi.

4.4 Architektúra hlavnej zostavy Wave Roveru

Zostava sa skladá z podvozku Wave Rover, Raspberry Pi 5, motorového HATu a pripojených senzorov. Z pohľadu architektúry je riešenie rozdelené do troch vrstiev. Prvú tvorí senzorická vrstva zahŕňajúca kameru, LiDAR a IMU. Druhú predstavuje riadiaca vrstva, v ktorej sa nachádza ovládanie motorov, korekčné algoritmy a autonómne režimy pohybu. Tretou je integračná vrstva, ktorá zabezpečuje spúšťanie systému, prepínanie režimov, webové rozhranie a prevádzkové služby.

Rozdelenie systému na vrstvy oddeľuje zber a publikovanie senzorických dát od samotnej logiky pohybu. Samostatné uzly pre LiDAR alebo kameru sú úzko špecializované a ľahko vymeniteľné, zatiaľ čo DriveNode spolu s jednotlivými režimami (wander, jednoduchá bodová navigácia alebo sledovanie objektu) pracujú nad jednotným rozhraním.

Takéto rozdelenie umožňuje oddeliť spracovanie senzorických dát od riadiacej logiky a zároveň zjednodušuje rozšírenie systému. Nové senzory alebo algoritmy je potom možné pridať ako samostatné uzly bez zásahu do existujúcich častí systému.

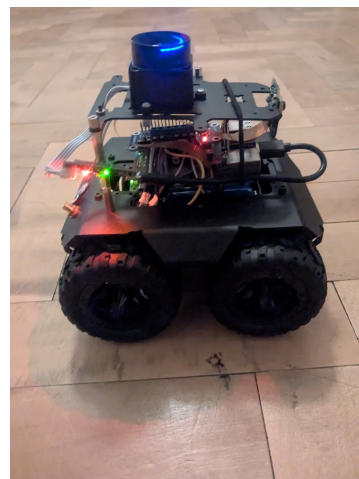
Alternatívne by sa dalo mať aj jeden veľký monolitický uzol, ktorý by naraz spracovával senzory, plánovanie aj riadenie motora. Také riešenie by však bolo postupom času ťažšie na testovanie, rozširovanie a rýchlo by v ňom vznikali problémy.



Pohľad spredu



Šikmý pohľad



Pohľad z boku

Obr. 4.2: Fyzická zostava platformy Wave Rover použitá pri hlavnej implementácii

4.5 Manuálne ovládanie, motorový uzol a kamera

Pred autonómnymi režimami a korekciou smeru je potrebné mať spoľahlivý základ: manuálne riadenie, jednotný výstup na motory a stabilný obraz z kamery. V tejto časti popisujem, ako tieto tri bloky spolu nadväzujú a aké ROS 2 rozhrania medzi nimi tečú.

4.5.1 Manuálne ovládanie a motorový uzol

Základným režimom práce robota je manuálne ovládanie. Na tento účel bol použitý štandardný uzol `teleop_twist_keyboard`, ktorý prevádza stlačenia kláves na správy typu `geometry_msgs/msg/Twist`. Tieto správy predstavujú v ROS 2 bežný spôsob zadávania lineárnej a uhlovej rýchlosti.

Samotný pohon robota zabezpečuje `DriveNode`, ktorý predstavuje jediný výstup smerom k motorom. Jeho úlohou je prijímať príkazy typu `Twist`, previesť ich na rýchlosti ľavej a pravej strany podvozku a následne generovať PWM signály pre motor HAT.

Pri diferenciálnom podvozku sa otáčanie dosahuje rozdielom rýchlosti ľavej a pravej strany. Ak sa jedna strana zrýchli a druhá spomalí, vznikne moment, ktorý robota točí okolo zvislej osi. V `teleop` režime sa táto idea používa tak, že zložka `fb` (*forward/back*) nastavuje spoločný pohon oboch strán (vpred alebo vzad), zatiaľ čo `tr` (*turn*) pridáva k ľavej strane a uberať z pravej (alebo naopak), čím sa mení pomer krútiacich momentov a robot sa stáča.

```
1 target_l_ = std::clamp(-fb + tr, -1.0, 1.0);  
2 target_r_ = std::clamp(-fb - tr, -1.0, 1.0);
```

Funkcia `std::clamp(..., -1.0, 1.0)` obmedzuje výsledné hodnoty do rozsahu, ktorý očakáva motorový ovládač. Bez tohto orezania by kombinácia zložiek pri prudkých vstupoch z klávesnice alebo webového rozhrania mohla prekročiť fyzikálne limity PWM signálu.

Pre klasický model diferenciálneho podvozku [13] sa pohyb robota rozkladá na lineárnu rýchlosť v a uhlovú rýchlosť ω . Tieto veličiny sa následne transformujú na rýchlosti ľavého a pravého kolesa podľa rozchodu kolies b v štandardnom tvare unicycle modelu. Lineárna zložka ovplyvňuje obe kolesá rovnako, zatiaľ čo uhlová zložka spôsobuje rozdiel ich rýchlostí, čím vzniká rotácia robota.

$$v_L = v - \omega \frac{b}{2}, \quad v_R = v + \omega \frac{b}{2}. \quad (4.1)$$

Ďalší krok v `drive_node.cpp` už nie je súčasťou samotného kinematického rozkladu a preto ju podrobnejšie rozoberiem v časti 4.6.

Webové rozhranie predstavuje paralelnú možnosť manuálneho ovládania. Pomocou `rosbridge` a knižnice `roslibjs` môže používateľ zadávať rovnaké príkazy z webového prehliadača.

4.5.2 Obraz z kamery pre teleop a vizuálnu korekciu

Kamera plní v systéme dve hlavné úlohy. Prvou je poskytovanie živého obrazu pri diaľkovom ovládaní robota. Druhou je poskytovanie vstupu pre algoritmy vizuálnej korekcie pohybu. (Dodatočne neskôr slúži aj na detekciu a sledovanie objektu.)

Po zapnutí modulu v launch konfigurácii je na Raspberry Pi spustený uzol `camera_ros`. Uzol publikuje štandardné ROS témy s obrazom, kamerovou kalibráciou a komprimovaným streamom.

Živý náhľad v diaľkovom webovom rozhraní

Pre náhľad z kamery v prehliadači sa neukázalo ako spoľahlivé prenášať každý JPEG rámec cez `rosbridge`, pretože WebSocket nad TCP a serializácia správ zvyšovali oneskorenie aj záťaž systému. Preto je v `runtime_stack.launch.py` predvolene zapnuté `use_webrtc_camera:=true`, pričom obraz sa odoberá z ROS 2 a prenáša do prehliadača cez WebRTC.

Ten istý obrazový tok je súčasne využívaný aj uzlom optického toku, ktorý prejdeme v neskorších sekciách.

4.6 Korekcia priameho smeru jazdy bez enkodérov

Pri testovacích jazdách sa už pri bežnej jazde vpred rýchlo ukáže, že udržať smer nie je samozrejmosťou. Aj keď sú obom stranám motorov posielané rovnaké príkazy, robot sa začne mierne stáčať na jednu stranu. Dôvodov môže byť viac: rozdielne vlastnosti motorov, nerovnosti povrchu, odlišný valivý odpor kolies, drobná mechanická asymetria alebo krátkodobé prešmykovanie kolies.

Pri platformách vybavených enkodérmi sa podobný problém zvyčajne rieši spätnou väzbou z otáčok kolies. V tomto prípade však boli použité motory bez enkodérov, takže nebolo možné priamo porovnávať skutočnú rýchlosť ľavej a pravej strany. Preto bolo nutné hľadať inú formu spätnej väzby, podľa ktorej by sa dalo určiť, či sa robot pri jazde vpred nechcene odchyľuje od smeru jazdy.

V implementácii boli overované dve metódy. Prvá využíva IMU, konkrétne uhlovú rýchlosť okolo zvislej osi. Druhá vychádza z optického toku v obraze z kamery. Obe metódy sa snažia zachytiť ten istý jav, teda odchýlku od priameho smeru, ale vychádzajú z iného typu merania.

4.6.1 MPU-6050 na Arduino Nano a USB most

Pôvodne bol senzor MPU-6050 plánovaný na spoločnej I²C zbernici spolu s motorovým HATom. Pri spustení sa však pri súčasnom riadení motorov a čítaní IMU objavovali nestability komunikácie, výpadky zariadení a nekorektné transakcie na zbernici. Keďže išlo o kritický bod celej architektúry, bolo zvolené hardvérové oddelenie IMU od tejto zbernice.

Senzor MPU-6050 bol preto pripojený k samostatnému mikrokontroléru Arduino Nano, ktorý vykonáva lokálne čítanie dát cez vlastnú I²C zbernicu. Firmware na Nano po inicializácii senzora vykoná kalibráciu gyroskopu a periodicky odosiela dáta po USB. Na Raspberry Pi ich skript `imu_serial_fusion_bridge.py` parsuje a zostavuje z nich správy `sensor_msgs/msg/Imu`.

4.6.2 Korekcia driftu podľa IMU

Cieľom nie je odhad absolútneho azimutu, ale zistenie, či sa podvozok pri jazde vpred neotáča. Pri rovnej jazde má byť ω_z z gyroskopu blízko nuly. Téma `/imu` vzniká podľa časti 4.6.1. V moste sa pritom využíva aj komplementárny filter s akcelerometrom na stabilnejší odhad orientácie. Do regulátora v `DriveNode` však vstupuje predovšetkým priamo gyroskopická zložka ω_z .

PID regulácia v motorovom uzle

Korekcia je implementovaná v `DriveNode`. Žiadaná hodnota je $\omega_{\text{ref}} = 0$, takže chyba regulácie je $e_k = -\omega_{z,k}$. PID regulátor [14] následne určuje korekciu ako kombináciu troch zložiek: okamžitej (proporcionálnej), kumulatívnej (integračnej) a predikčnej (derivačnej), pričom každá z nich ovplyvňuje správanie robota pri odchýlke od priamej jazdy.

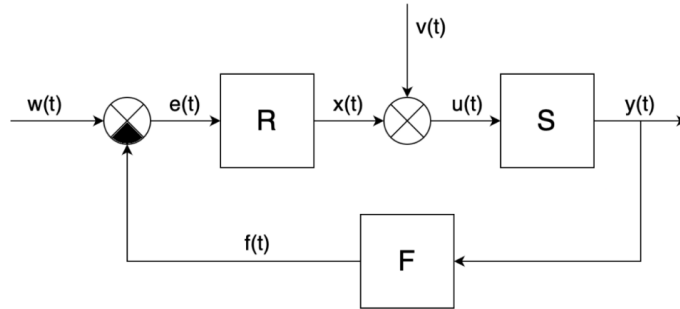
V spojitom čase ho možno chápať ako kombináciu proporcionálnej, integračnej a derivačnej zložky. Pre potreby digitálnej implementácie v tomto systéme bol tento vzťah aproximovaný do diskrétného tvaru:

$$u_k = K_p e_k + K_i \sum_{j=0}^k e_j \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}, \quad (4.2)$$

kde u_k je výsledná korekcia aplikovaná diferenciálne na pohon – jedna strana sa zrýchľuje a druhá spomaľuje.

Proporcionálna zložka reaguje okamžite na aktuálnu odchýlku, integračná odstraňuje dlhodobý drift a derivačná tlmí rýchle zmeny, čím stabilizuje reakciu systému. Koeficienty K_p , K_i , K_d a obmedzenie integrátora sú nastaviteľné parametre uzla.

V `drive_node.cpp` je pred aplikáciou PID na ω_z použitá mŕtva zóna a exponenciálne vyhladenie signálu. Podmienka `going_straight` vypína korekciu pri zastavení, ostrých zatáčkach, veľkom rozdiel rýchlostí kolies alebo pri úmyslnom manévri, aby nezasahovala do zámerného riadenia.



Obr. 4.3: Zovšeobecnená spätnoväzbová slučka: žiadaná hodnota $w(t)$, chyba $e(t)$, regulátor R , signál z regulátora $x(t)$, porucha $v(t)$, vstup do sústavy $u(t)$, sústava S , výstup $y(t)$, spätná väzba cez F .

Použitý bol jednoduchý exponenciálny filter (EMA), ktorý zodpovedá bežnému tvaru jednopólového rekurzívneho vyhladenia a obmedzuje šum a skoky:

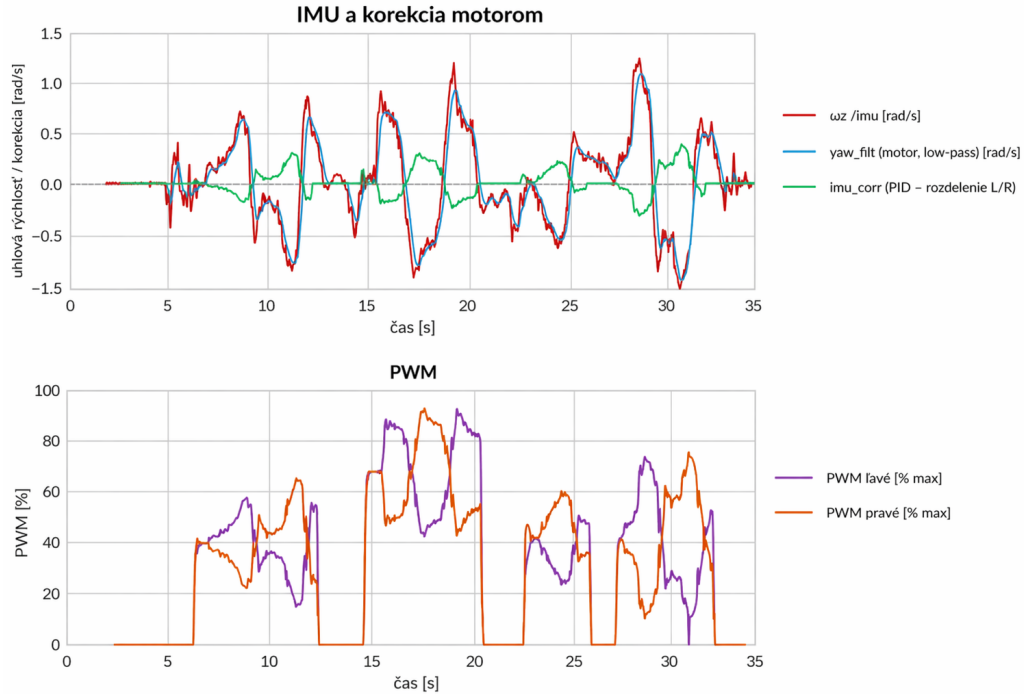
$$y_k = \alpha x_k + (1 - \alpha) y_{k-1}, \quad \alpha \in (0, 1), \quad (4.3)$$

pričom menšie α vedie k plynulejšej, ale pomalšej odozve. V implementácii sa používa na vyhladenie cieľových rýchlostí kolies (`smooth_alpha`) aj na filtrovanie ω_z pred vstupom do PID (`imu_yaw_lowpass_alpha`).

Výstup PID regulátora sa následne aplikuje diferenciálne na pohon – z jednej strany sa odčíta a na druhej pripočíta, pričom výsledné hodnoty sa obmedzia do rozsahu $[-1, 1]$ pomocou `std::clamp`, aby zostali v medziach motorového ovládača.

Význam a využitie IMU korekcie

Metóda je vhodná pri slabom osvetlení a bez závislosti na textúre podlahy. Kvalita závisí od tlmenia šumu a biasu gyroskopu.



Obr. 4.4: Ukážka vzťahu medzi meranou uhlovou rýchlosťou a korekciou pohonu

4.6.3 Korekcia driftu podľa optického toku

Druhou skúšanou metódou bola korekcia podľa optického toku z kamery³. Na rozdiel od IMU tu nejde o meranie rotácie priamo zo senzora, ale o pozorovanie toho, ako sa mení obraz pri pohybe robota podľa modelu Lucas–Kanade [15]. Ak robot ide rovno dopredu a prostredie pred ním je približne statické, horizontálny posun obrazu by sa mal dlhodobo držať okolo nuly. Ak sa začne nechcene stáčať alebo uhýbať do strany, v obraze sa to prejaví ako postupný bočný posun sledovaných bodov. V implementácii sa z neho po priemerovaní a normalizácii vytvorí malá úprava `angular.z` v `Twist`, ktorá dopĺňa teleop rovnako ako pri IMU korekcii, ale z vizuálneho vstupu.

Teoretický základ

Optický tok opisuje pohyb obrazových bodov medzi dvoma po sebe idúcimi snímkami. Zmeny polohy bodov umožňujú odhadnúť pohyb objektov alebo kamery v scéne.

Metóda Lucas–Kanade vychádza z predpokladu konštantnosti jasu bodu medzi snímkami [15]: $I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$. Po linearizácii vznikne vzťah

$$I_x u + I_y v + I_t = 0, \quad (4.4)$$

³OpenCV tutorial k optickému toku: https://docs.opencv.org/3.4/d4/dee/tutorial_optical_flow.html.

ktorý sa rieši pre rýchlosť pohybu (u, v) v malom okolí bodu metódou najmenších štvorcov.

V implementácii sa najprv vyberú stabilné body pomocou Shi–Tomasi detektora (`goodFeaturesToTrack`) [16], ktorý preferuje rohy a výrazné hrany s vysokou zmenou intenzity v dvoch smeroch. Tieto body sa následne sledujú medzi snímkami pomocou `calcOpticalFlowPyrLK`.

Výsledkom sú vektory posunu jednotlivých bodov, z ktorých sa pre riadenie využíva ich priemerná horizontálna zložka. Tá reprezentuje odchýlku robota od priamej jazdy a po normalizácii sa používa ako jemná korekcia uhlovej rýchlosti `angular.z`.

Spracovanie v uzle `OpticalFlowNode`

Uzol priebežne odoberá obraz z kamery a sleduje aktuálny teleoperačný príkaz, ku ktorému môže podľa potreby pridať korekciu smeru.

Každá snímka sa najprv dekoduje a prevedie do odtieňov sivej, keďže pre výpočet optického toku sú rozhodujúce zmeny jasů. Následne sa obraz zmenší, aby bol výpočet dostatočne rýchly.

Na predchádzajúcej snímke sa vyberú body vhodné na sledovanie pomocou metódy Shi–Tomasi (`goodFeaturesToTrack`) [16]. Tieto body sa následne sledujú medzi snímkami pomocou metódy Lucas–Kanade (`calcOpticalFlowPyrLK`) [15].

Výpočet prebieha len v lokálnom okolí každého bodu, čo umožňuje efektívne spracovanie v reálnom čase.



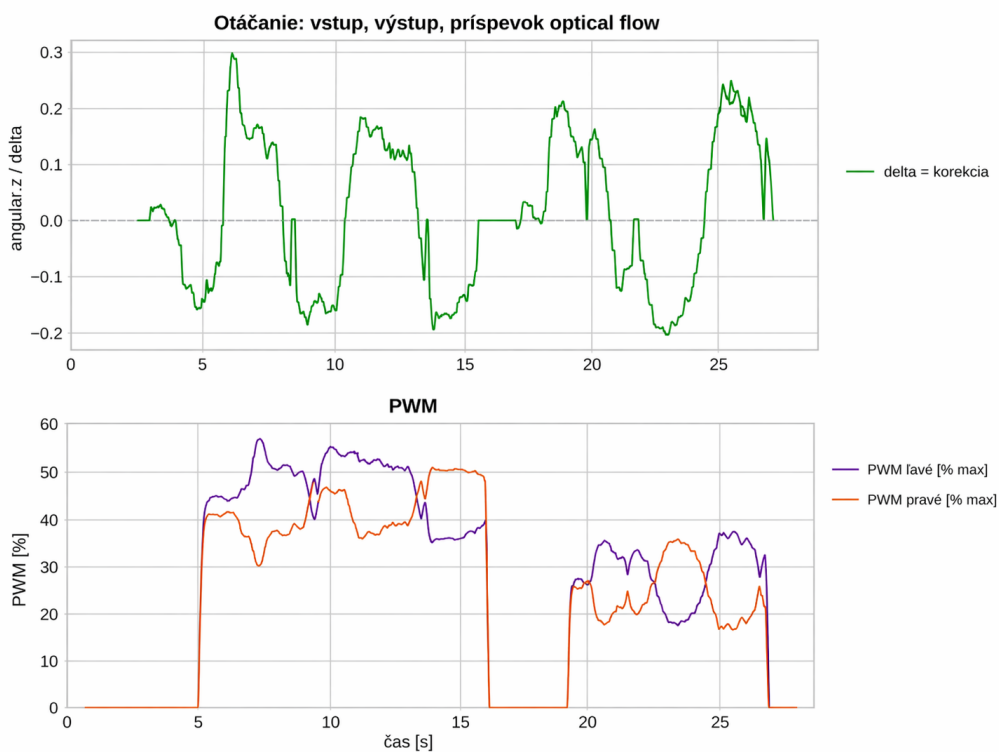
Obr. 4.5: Sledované body a vektory posunu pri výpočte optického toku (ukážka z ladenia uzla)

```
1 prev_pts = cv2.goodFeaturesToTrack(prev_gray, maxCorners=100,  
2                                     qualityLevel=0.01, minDistance=10)  
3 curr_pts, status, _ = cv2.calcOpticalFlowPyrLK(prev_gray, frame,  
4                                                  prev_pts, None)
```

Zo všetkých úspešne sledovaných bodov sa počíta priemerný horizontálny posun. Ak sa napríklad väčšina bodov posúva v obraze doľava, znamená to, že robot má tendenciu uhýbať doprava. Znamienko a veľkosť priemernej hodnoty tak poskytujú jednoduchý odhad odchýlky od priameho smeru.

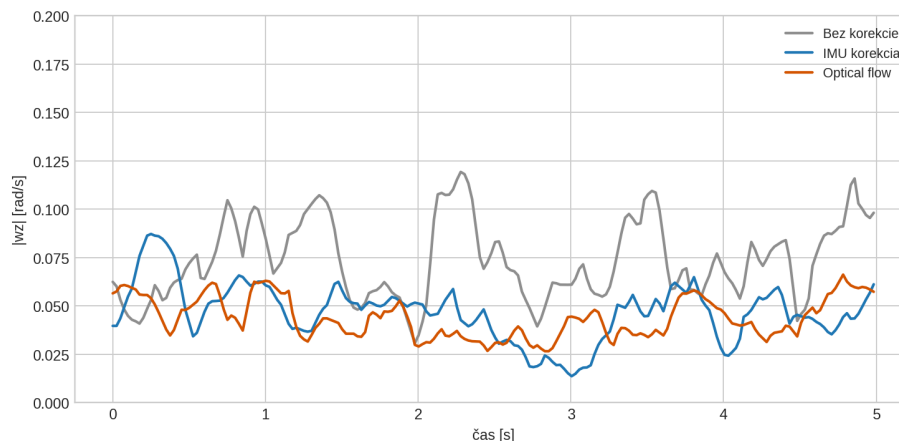
Priemerovanie viacerých bodov má výhodu v tom, že výsledok je odolnejší voči chybným meraniam jednotlivých bodov. Ak sa niektorý bod stratí, zakryje alebo je vyhodnotený nepresne, ostatné body jeho vplyv zmiernia.

Táto hodnota sa následne normalizuje podľa šírky obrazu, aby výsledok nebol závislý od použitého rozlíšenia. Po vynásobení nastaviteľným ziskom vznikne malá korekcia, ktorá sa pripočíta k zložke `angular.z`. Robot tak dostane povel na dorovnanie smeru jazdy.



Obr. 4.6: Priebeh korekcie uhlovej zložky riadenia a súvisiacich signálov v čase (horný graf: `angular.z` a príspevok optického toku; dolný graf: PWM po spracovaní v `drive_node`)

V grafoch tejto časti používam uhlovú rýchlosť okolo zvislej osi robota v jednotkách rad/s. Pri ideálnej rovnej jazde má byť jej absolútna hodnota čo najmenšia.



Obr. 4.7: Porovnanie odchýlky jazdy počas 5-sekundového úseku pre tri režimy korekcie, pričom zobrazené priebehy predstavujú priemery z viacerých jazd.

4.7 Autonómne blúdenie (wander)

Jedným z cieľov implementácie bolo doplniť platformu o autonómny režim pohybu. Na tento účel som implementoval režim *wander* založený na reaktívnom vyhýbaní sa prekážkam podľa aktuálnych dát z LiDARu.

Senzor LD19 je obsluhovaný driverom `ldlidar_ros2`, ktorý publikuje laserový sken na tému `/scan`. Namiesto spracovania celého skenu zložitým plánovačom je priestor okolo robota rozdelený na tri základné sektory: oblasť vpredu, oblasť vľavo a oblasť vpravo.

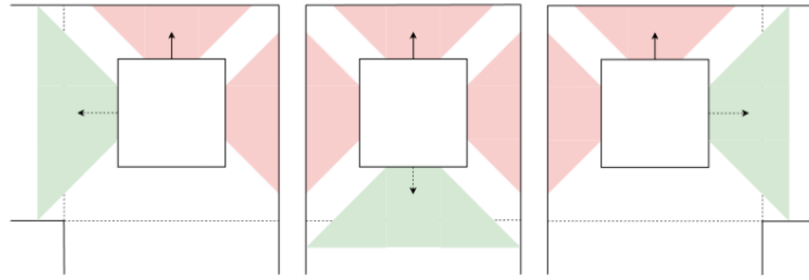
Pre každý sektor sa z poľa `ranges` vyhodnocuje minimálna platná vzdialenosť. Neplatné merania alebo extrémne malé hodnoty sú ignorované.

```

1      def _sector_min(self, ranges, angle_min, angle_inc,
2                      lo_deg, hi_deg, rotation_rad):
3          best = math.inf
4          for i, d in enumerate(ranges):
5              if not math.isfinite(d) or d < 0.02:
6                  continue
7              ...
8              if lo <= a_robot <= hi:
9                  best = min(best, d)
10         return best

```

Takto získané tri hodnoty predstavujú zjednodušený model voľného priestoru v okolí robota a slúžia ako vstup pre rozhodovaciu logiku autonómneho režimu.

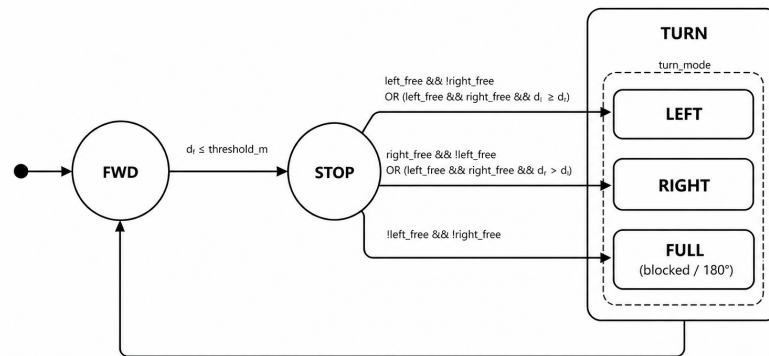


Obr. 4.8: Rozhodovacia logika autonómneho blúdenia podľa mínim v sektoroch

Stavový automat wanderu.

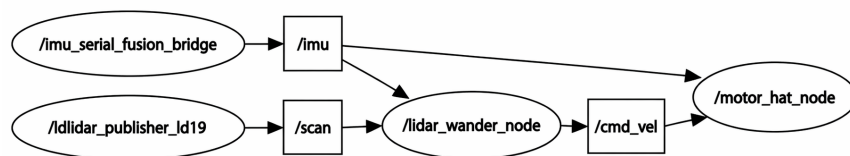
Riadenie je realizované ako jednoduchý stavový automat so stavmi jazdy vpred a otáčania. V predvolenom stave robot pokračuje dopredu konštantnou rýchlosťou `forward_speed`, pokiaľ je priestor pred ním voľný a minimálna vzdialenosť v prednom sektore je väčšia než nastavený prah `threshold_m`.

Ak sa pred robotom objaví prekážka bližšie než zvolený limit, systém porovná voľný priestor v ľavom a pravom sektore. Následne vyberie vhodný smer otočenia. Ak je napríklad voľnejšia ľavá strana, robot sa otočí doľava. Ak sú obe strany podobne voľné, zvolí sa výhodnejší smer podľa väčšej vzdialenosti. V prípade, že sú blokové obe strany, vykoná sa otočenie o 180° .



Obr. 4.9: Stavový automat uzla `lidar_wander_node` podľa implementácie v `lidar_wander.py`

Na obr. 4.10 je ukážka behu režimu v nástroji `rqt`: laserový sken a súvisiace témy počas autonómneho blúdenia.



Obr. 4.10: Vizualizácia dát z LiDARu v nástroji `rqt` počas autonómneho bludenia

Počas otáčania robot generuje uhlovú rýchlosť okolo osi z . Odhad natočenia počas manévru sa v `lidar_wander.py` robí integráciou gyroskopickej zložky ω_z z IMU.

4.8 Mapovanie a jednoduchá bodová navigácia

Okrem reaktívneho wanderu som chcel ukázať aj prácu s mapou a cieľom v súradniciach `map`. Sekcia najprv stručne predstavuje 2D SLAM a distribuované spustenie náročnejších uzlov, potom jednoduchú bodovú navigáciu bez plného Nav2 stacku na reálnom podvozku.

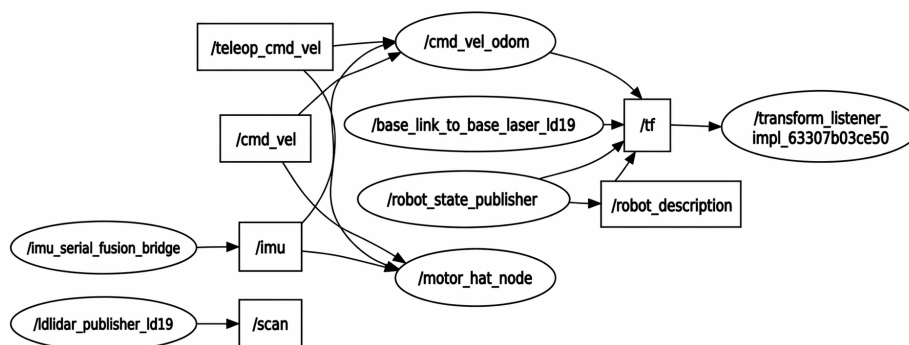
Mapovanie prostredia (SLAM)

Dôležitou súčasťou autonómneho pohybu Wave Roveru je schopnosť vytvárať mapu neznámeho prostredia a zároveň odhadovať vlastnú polohu v tejto mape. Táto úloha je známa ako *SLAM* (*Simultaneous Localization and Mapping*). Robot pri nej postupne získava merania zo senzorov, z ktorých buduje mapu okolia a súčasne spresňuje svoj odhad pozície.

Na riešenie mapovania bol použitý balík `slam_toolbox`, ktorý patrí medzi štandardné nástroje pre 2D SLAM v prostredí ROS 2. Zvolený bol pre jeho ľahkú integráciu s ostatnými časťami zostavy a priamu nadväznosť na navigačné úlohy.

Balík pracuje predovšetkým s 2D LiDAR dátami a transformáciami medzi rámcami robota. Mapa je reprezentovaná *occupancy gridom*, čo je pravidelná mriežka buniek, kde každej bunke zodpovedá odhad pravdepodobnosti obsadenia. Nové LiDAR skeny sa neukladajú len podľa odometrie, ale zarovnávajú sa voči už existujúcej mape, čím sa znižuje drift odhadu polohy.

Súčasťou SLAM je aj *uzatváranie slučiek*. Ak sa robot po čase vráti do už navštívenej oblasti, systém rozpozná známu časť prostredia a opraví nahromadenú chybu v trajektórii aj v mape. Vďaka tomu zostáva výsledná mapa konzistentnejšia aj pri dlhšom pohybe.



Obr. 4.11: Prepojenie uzlov pri manuálnom režime a súčasne spustenom mapovaní

Distribúované nasadenie

Výpočtovo náročnejšie časti systému, najmä SLAM a plynulá vizualizácia v RViz, výrazne zatažovali Raspberry Pi, čo sa prejavovalo na stabilite ostatných procesov. Z tohto dôvodu som tieto komponenty spúšťal na výkonnejšom externom počítači.

Na Raspberry Pi zostáva *runtime stack* zabalený v `runtime_stack.launch.py`. Pri zapnutí mapovania s parametrom `use_offboard_slam:=true` sa uzol `slam_toolbox` nespúšťa na Raspberry Pi, ale beží na externom počítači pomocou launch súboru `slam_remote_pc.launch.py`. Na tomto počítači je zároveň spustený aj RViz, ktorý slúži na vizualizáciu mapy a aktuálneho stavu lokalizácie.

Jednoduchá navigácia

V systéme je okrem režimu *wander* implementovaný aj jednoduchý navigačný režim *simple nav*, aby bolo možné aspoň sčasti ukázať základné princípy riadenia bez komplexného Nav2 stacku.

Cieľ sa zadáva v RViz pomocou nástroja *2D Goal Pose* a prichádza ako správa `geometry_msgs/msg/PoseStamped` na tému `/goal_pose`. Poloha cieľa aj aktuálna poloha robota sa získavajú z transformácie `map` $\rightarrow$ `base_link` cez SLAM.

Po prijatí cieľa sa robot najprv otočí smerom k nemu a následne sa k nemu pohybuje priamo dopredu. Rýchlosti sa posielajú ako `geometry_msgs/msg/Twist` na `/cmd_vel`.

V simulácii bol použitý plnohodnotný stack Nav2, ktorý obsahuje globálne aj lokálne plánovanie. V reálnom nasadení sa však ukázalo, že priame riadenie cez Nav2 a generované rýchlosti nebolo dostatočne stabilné. Preto bola implementovaná primitívnejšia verzia, ktorá rieši iba presun medzi bodmi bez komplexného plánovania.

4.9 Sledovanie farebnej lopty

V tejto časti opisujem sledovanie jednofarebného objektu (v testoch pomaranč) pomocou kamery a následné priblíženie robota. Z obrazu sa vždy vyberá len horizontálna odchýlka cieľa od stredu záberu a veľkosť jeho projekcie v pixeloch. Na základe týchto dvoch hodnôt regulátor generuje príkaz `Twist`, ktorý sa publikuje na tému `/cmd_vel` podobne, ako to bolo u korekcii jazdy.

Postup detekcie vychádza z farebnej segmentácie v HSV, ošetrenia binárnej masky a extrakcie kontúr; konkrétne funkcie a operácie pri jednotlivých krokoch odkazujem na príslušné časti dokumentácie OpenCV nižšie. V implementácii sa používa vlastný výber kandidáta z kontúr. Ako východiskový nápad poslúžil verejný ROS 2 balík Josha Newansa⁴, no v navrhnutom riešení je nad ním postavená vlastná riadiaca vrstva s rozhodovacou logikou sledovania a spôsobom spúšťania.

Tri vrstvy Architektúry

- `detector.py` — obsahuje triedu `OrangeDetector`, ktorá vracia štruktúru `Detection` (`x_err`, `radius_px`, `confidence`).
- `controller.py` — trieda `BallController`, ktorá spracuje detekciu cez `update(det, has_frame)` a vráti `Twist` spolu so stavom.
- `action_node.py` — `BallFollowerNode`, ktorý zabezpečuje príjem obrazu, publikovanie dôvery v správnu detekciu, debug obraz a akčný server typu `FollowBall`.

Detekcia v `OrangeDetector`

Detekcia začína funkciou `_make_mask`. Najprv sa vstupný obraz zmenší na polovicu, aby sa znížila výpočtová náročnosť, pričom sa stále zachová dostatok informácie na spoľahlivú detekciu objektu.

Následne sa obraz prevedie z farebného priestoru BGR do HSV, ktorý je pre farebnú segmentáciu vhodnejší, pretože oddelí informáciu o farbe od jasú. Na základe nastavených intervalov pre oranžovú farbu (`orange_h_min/max`, `s_*`, `v_*`) sa pomocou funkcie `cv2.inRange` [17] vytvorí binárna maska, kde sú pixely patriace lopte biele a zvyšok čierny.

Takto vzniknutá maska však obsahuje aj drobné šumy a rušenie, preto sa ďalej čistí morfológickými operáciami (**opening** `MORPH_OPEN`, **closing** `MORPH_CLOSE`) podľa [18]: opening je zložením erozie a dilatácie a odstraňuje drobné artefakty, closing je zložením dilatácie a erozie a vyplňa malé dierky v oblasti objektu. V implementácii najprv nasleduje opening, potom closing.

Z výslednej masky sa následne extrahujú kontúry pomocou `findContours(..., RETR_EXTERNAL)` [19], pričom sa berú len vonkajšie obrysy objektov. Veľmi malé kontúry sa zahadzujú, pretože zvyčajne znamenajú šum alebo odrazy.

Pre každú zostávajúcu kontúru sa vyhodnocuje, či ide skutočne o guľu podľa rôznych vlastností. Na vyhodnotenie kandidátnych kontúr sa v implementácii používa vlastná skórovacia heuristika. Z geometrických vlastností kontúry sa zostaví veličina spoľahlivosti:

$$\text{conf} = \text{clip}(0,40\,c + 0,35\,s + 0,15\,a + 0,10\,\text{clip}(f, 0, 1)),$$

kde c predstavuje kruhovitost', s soliditu, f vyplnenie kruhu a a normalizovanú plochu vzhľadom na veľkosť masky. Ak je výsledná hodnota menšia než prah `min_confidence`, kontúra sa ignoruje. Z platných kandidátov sa následne vyberie tá s najvyššou spoľahlivosťou.

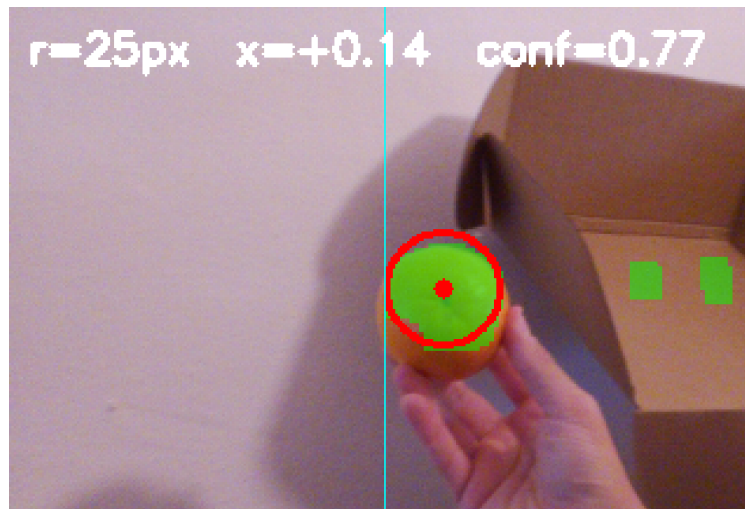
Pozícia lopty sa určuje až na pôvodnom rozlíšení obrazu. Zo stredu detegovaného kruhu sa v implementácii počíta normalizovaná horizontálna odchýlka od stredu, kde w je šírka obrazu:

⁴Repozitár `ball_tracker`: https://github.com/joshnewans/ball_tracker.

$$x_err = \frac{c_x - w/2}{w/2},$$

Keďže detekcia môže občas preskočiť na iný objekt alebo šum, systém kontroluje, či sa poloha stredu medzi snímkami príliš nezmenila. Ak je skok väčší než povolený limit, detekcia sa zahodí a označí sa ako miss. Ak sa takáto situácia opakuje dlhšie, než je nastavený čas `jump_reset_sec`, história sledovania sa resetuje, aby sa systém znovu stabilizoval.

Pre ladenie som generoval vizuálny výstup, ktorý zobrazuje masku prekrytú na pôvodnom obraze, stred obrazu, detegovaný kruh a aktuálnu pozíciu lopty. Dopĺňajú sa aj hodnoty polomeru, odchýlky `x_err` a spoľahlivosti `conf`.



Obr. 4.12: Debug v `rqt_image_view` na `/ball_follower/debug_image/compressed`: maska, polomer a horizontálna odchýlka.

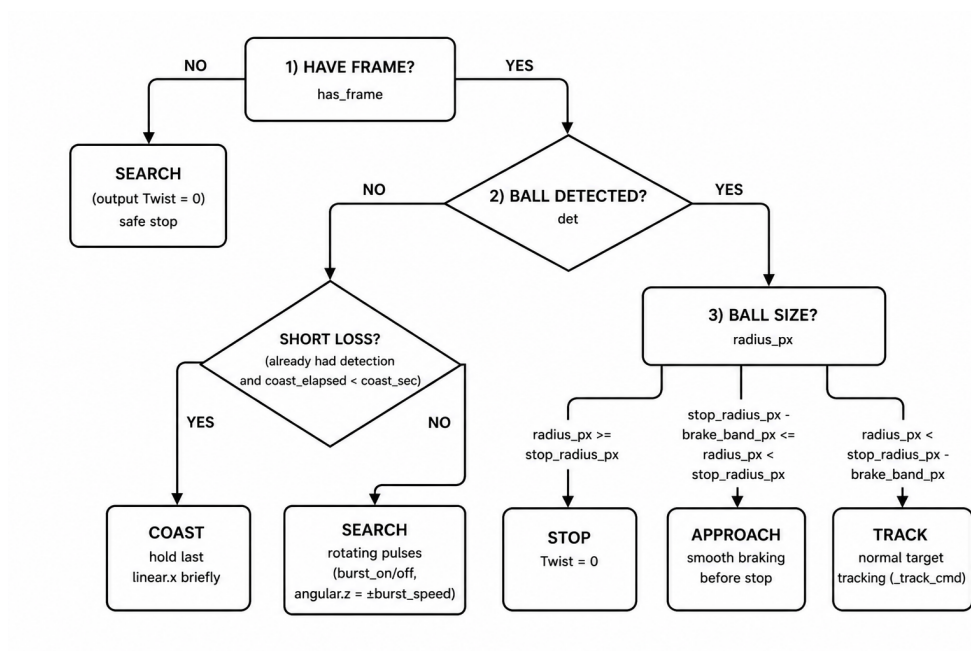
Rozhodovacia logika `BallController`

Riadenie sledovania nie je v kóde zapísané ako tabuľka prechodov klasického stavového automatu, ale ako *deterministické priradenie režimu* v každom volaní `update()`: z aktuálneho vstupu (snímka, detekcia, polomer, čas od poslednej detekcie) sa vypočíta príkaz `Twist` a súčasne sa označí jedna z hodnôt výčtu `State` (`SEARCH`, `TRACK`, `APPROACH`, `COAST`, `STOP`) najmä na ladenie a prehľad. Správanie je teda najlepšie chápať ako rozhodovací strom; hodnoty `State` zodpovedajú vetvám tohto stromu, nie explicitnej vnútornej reprezentácii v zmysle *aktuálny stav + udalosť → nový stav*.

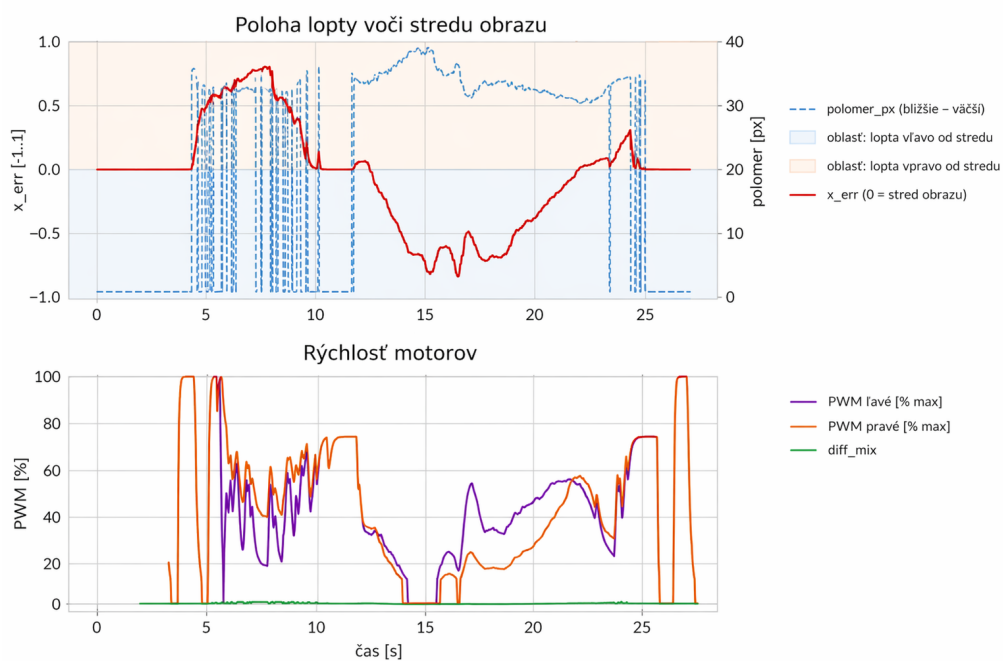
Ak nie je k dispozícii obraz, výstup je bezpečná zástava a režim `SEARCH`. Pri krátkodobej strate detekcie sa použije `COAST` s poslednou zapamätanou doprednou rýchlosťou. Ak je objekt v zábere dostatočne veľký, aktivuje sa `STOP`.

Inak robot cieľ sleduje v režime `TRACK`; pri približovaní v pásme pred zastavením sa rýchlosť a zatačanie zmierňujú (`APPROACH`) podľa `stop_radius_px` a `brake_band_px`. Obr. 4.13 zobrazuje rovnakú logiku vo forme toku rozhodnutí pre jedno volanie `update()`.

Vplyv vzdialenosti objektu sa zohľadňuje škálovaním podľa jeho polomeru, čím sa zvyšuje citlivosť riadenia pri bližšom objekte. Výsledný príkaz `Twist` je následne obmedzený na bezpečný rozsah a vyhladený pomocou EMA filtra.



Obr. 4.13: Rozhodovací diagram režimov sledovania lopty (BallController, metóda update()).

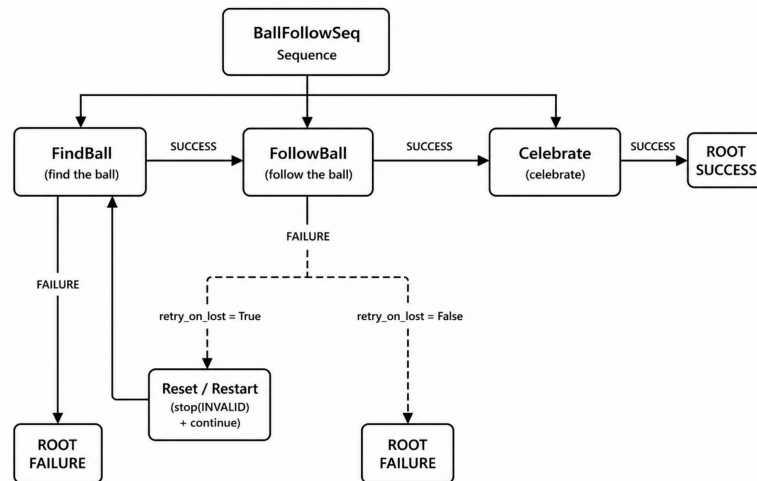


Obr. 4.14: Priebeh lineárnej a uhlovej rýchlosti počas sledovania objektu.

Behaviorálny strom ako orchestrátor

`BallController` rieši lokálnu rozhodovaciu logiku sledovania objektu, napríklad reakciu na stratu cieľa, zatiaľ čo behaviorálny strom riadi celkové správanie robota na vyššej úrovni.

Vyššiu logiku úlohy zabezpečuje uzol `bt_runner.py` využívajúci knižnicu `py_trees` [20]. Behaviorálny strom (Obr. 4.15) organizuje celý priebeh sledovania do troch na seba nadväzujúcich fáz: **FindBall**, **FollowBall** a **Celebrate**. Jednotlivé kroky opakovane využívajú ten istý akčný server `follow_ball`, ale s rôznymi parametrami cieľa.



Obr. 4.15: Strom uzla `ball_follow_bt_runner` (`ball_follow/bt_runner.py`)

Koreň stromu je sekvencia, takže po úspešnom dokončení kroku sa pokračuje ďalším bez opätovného spúšťania predchádzajúcich fáz. V prípade zlyhania sledovania môže strom skončiť chybou alebo sa celý scenár resetuje a začne znova od **FindBall**, v závislosti od parametra `retry_on_lost`.

Behaviorálny strom tak riadi makro priebeh úlohy a samotný regulátor `BallController` rieši krátkodobé výpadky detekcie na nižšej úrovni.

Akčný server `follow_ball`

Metóda `_execute_cb` v `action_node.py` obsluhuje vykonávanie cieľa. Po prijatí nastaví príznaky `_goal_busy` a `_active` a v pravidelnej slučke kontroluje zrušenie a podmienky úspechu alebo zlyhania.

Pre režim **FindBall** (`stop_when_found=True`) je kritérium že akcia skončí úspešne hneď, ako sa objaví čerstvá detekcia. V režime **FollowBall** môže byť zapnuté aj zlyhanie pri strate objektu.

Úspech pri sledovaní sa vyhodnocuje vtedy, keď je detekcia čerstvá a $\text{radius_px} \geq \text{stop_radius_px}$. Časová značka sa uloží do fronty `close_times`.

Implementácia a spustenie

Systém sa spustí pomocou `ball_follow_standalone.launch.py`, ktorý inicializuje motor, kameru, uzol `ball_follower` a `bt_runner`. Alternatívne sa dá použiť aj offboard spracovanie (`offboard_cv:=true`).

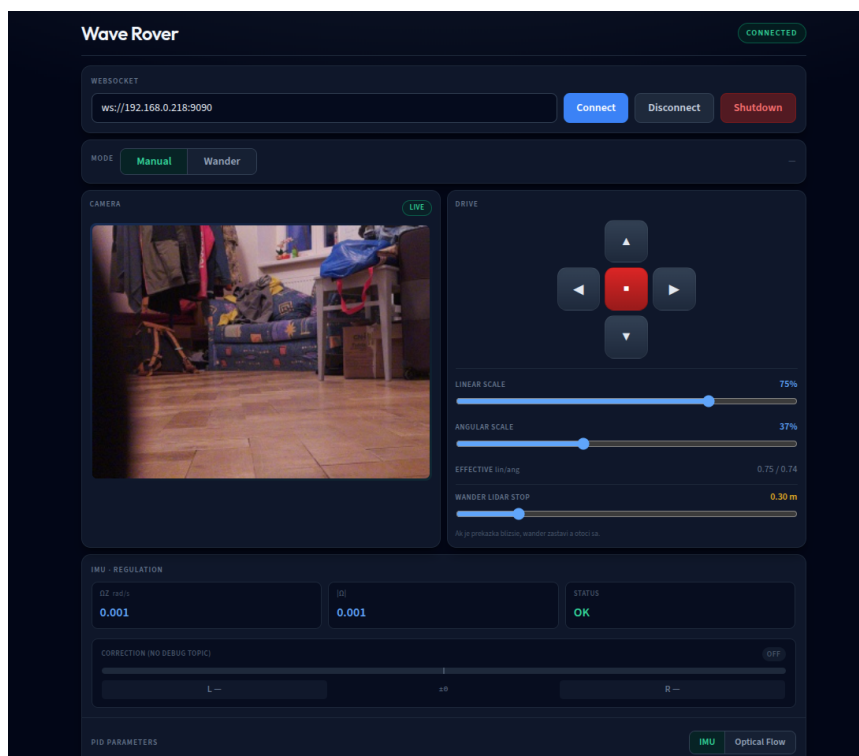
4.10 Runtime stack, webové rozhranie a prevádzkové služby

Webové rozhranie som do systému pridal, aby bolo jednoduché ovládať robota bez potreby priamej práce s ROS 2 nástrojmi.

4.10.1 Webové rozhranie

Frontend aplikácie je implementovaný ako jednostránkové rozhranie, ktoré komunikuje s ROS 2 prostredím prostredníctvom knižnice *roslibjs* [21] nad serverom *rosbridge_suite* [22].

Rozhranie poskytuje informáciu o stave pripojenia, živý obraz z kamery, tlačidlá pre pohyb robota, tlačidlá pre prepínanie režimov a parametrov regulácie a tlačítko na vzdialené vypnutie.



Obr. 4.16: Uživatelské rozhranie dostupné v prehliadači

4.10.2 Komunikačná vrstva.

Po nadviazaní WebSocket spojenia sa v klientovi vytvorí publisher na tému `/teleop_cmd_vel`. Pohybové príkazy sa odosielajú opakovane s frekvenciou 30 Hz.

Živý obraz z kamery v paneli rozhrania je riešený podľa časti 4.5.2.

4.10.3 Ovládanie pohybu.

Na zadávanie pohybu slúži smerový ovládač typu D-pad. Každé tlačidlo obsahuje definovanú lineárnu a uhlovú zložku pohybu. Na mobilných zariadeniach sú použité dotykové udalosti `touchstart` a `touchend`.

4.10.4 Runtime stack, prepínanie manuál / wander a graf uzlov

Na plynulé prepínanie medzi manuálnym ovládaním a autonómnym režimom som navrhol samostatný integračný uzol `robot_mode_switch`, ktorý zabezpečuje zmenu režimu za behu systému. Využíva klienta `AsyncParameterClient`, čím mení parametre iných bežiacich uzlov bez potreby ich reštartu.

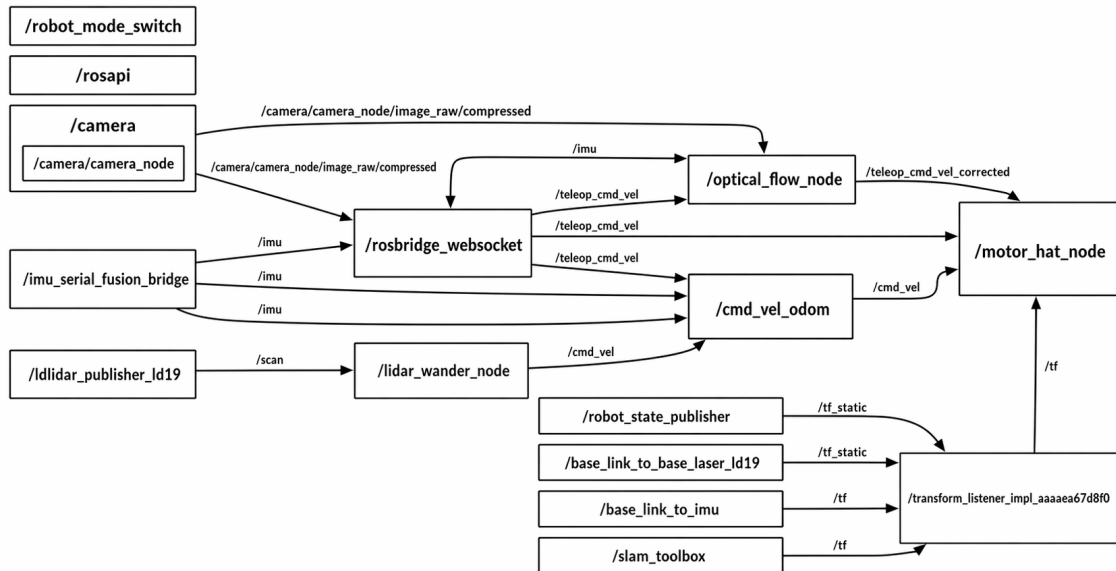
```
1     self._motor = _RemoteParamClient(self, "/motor_hat_node")
2     self._wander = _RemoteParamClient(self, "/lidar_wander_node")
3
4     fut = self._motor.set_parameters([
5         Parameter("control_mode",
6                 Parameter.Type.STRING,
7                 "manual")
8     ])
```

V ROS 2 sa dynamická rekonfigurácia rieši cez parametre uzlov. Uzol `robot_mode_switch` počas behu mení parametre ostatných uzlov pomocou `AsyncParameterClient`.

Ďalšie nastavenia je možné meniť cez argumenty launch súborov a YAML konfigurácie (napr. voľba portu IMU, zapnutie offboard spracovania alebo výber korekcie jazdy), pričom prehľad hlavných uzlov a prepínačov je uvedený v tab. 4.1.

4.10.5 Graf uzlov a dátových tokov.

Na obr. 4.17 je graf uzlov pri spustení `runtime_stack.launch.py`. Je z neho vidieť prepojenie motorového uzla, uzla blúdenia, prepínača režimov, senzorických driverov a webového rozhrania.



Obr. 4.17: Prepojenie hlavných uzlov, tém a služieb pri spustenej fyzickej platforme

4.10.6 Vzdialené vypnutie Raspberry Pi a tlačidlo Shutdown v používateľskom rozhraní

Pri používaní robota sa ukázalo ako praktické mať možnosť systém vypnúť na diaľku bez potreby pripájať klávesnicu alebo monitor.

Túto funkcionálnosť som implementoval v uzle `robot_mode_switch`, ktorý poskytuje službu `/waverover/shutdown`. Po prijatí požiadavky uzol najprv odošle potvrdenie a následne vykoná príkaz `sudo systemctl poweroff`.

4.10.7 Integrita súborového systému pri výpadku napájania

Pri mobilnej platforme som musel počítať aj s možnosťou náhleho odpojenia napájania. Z tohto dôvodu uprednostňujem korektné vypnutie systému cez službu `/waverover/shutdown`, ktoré znižuje riziko poškodenia dát.

Napriek tomu sa takýmto situáciám nedá úplne vyhnúť, preto je dôležité aj vhodné nastavenie operačného systému. Jednou z možností je použitie read-only systémového oddielu, pri ktorom sa počas bežnej prevádzky nezapíše na disk, čím sa zníži riziko poškodenia pri výpadku.

Ak treba zapisovať dáta, dá sa využiť overlay vrstva, napríklad v RAM. Riešenie sa potom navonok správa štandardne, ale zmeny sa neukladajú priamo do úložiska.

Ako doplnkové opatrenie sa dajú použiť aj odolnejšie pamäťové médiá [23], napríklad high-endurance alebo industrial microSD karty navrhnuté na časté prepisovanie dát a dlhodobú prevádzku. Oproti bežným kartám výrobcovia deklarujú vyššiu odolnosť voči zápisovým cyklom a vhodnosť pre dlhodobú prevádzku v náročnejších podmienkach. V tomto projekte sa mi však osvedčila aj bežná SD karta.

4.11 Overenie implementácie a prehľad uzlov

Po návrhu a implementácii jednotlivých častí systému bolo potrebné overiť, ako sa správajú pri reálnom použití. V tejto kapitole preto opisujem testovanie robota, jeho jednotlivých režimov a vzájomnej spolupráce uzlov v ROS 2.

Metodika testovania

Overenie systému prebiehalo praktickými jazdnými testami v domácom prostredí. Robot bol skúšaný na rôznych povrchoch a pri rôznych svetelných podmienkach. Cieľom bolo hlavne overiť, či jednotlivé režimy fungujú spoľahlivo v bežných podmienkach.

Počas testovania som správanie systému sledoval pomocou nástrojov `rqt_graph`, `rqt_image_view`, `rqt_plot` a ROS 2 CLI. Pomocou `rqt_graph` som kontroloval prepojenie uzlov a tém, `rqt_image_view` slúžil na sledovanie obrazových výstupov a debug streamov a `rqt_plot` pomáhal analyzovať priebeh veličín pri korekcii smeru.

Ladenie IMU korekcie a PID v motorovom uzle

Ladenie IMU korekcie prebiehalo iteratívne. Najprv som overil, či sa korekcia správne aktivuje, a potom som postupne doladzoval jednotlivé parametre. Pri rovnej jazde som riešil najmä filtráciu a mŕtvu zónu. Ak robot kmital okolo nuly, mierne som zvýšil `imu_deadband` alebo znížil `imu_yaw_lowpass_alpha`, čím sa odozva vyhladila.

Parametre PID regulátora som doladľoval postupne odhadom podľa pozorovania správania robota pri jazde.

Testovanie a ladenie optického toku

Optický tok som tiež ladil podľa správania robota a vizuálnej spätnej väzby. Ukázalo sa, že funguje spoľahlivo len pri dostatočnej textúre a osvetlení. Na jednofarebnom pozadí alebo pri slabom svetle bol počet sledovaných bodov nízky a odhad pohybu bol nespoľahlivý.

Parametre `correction_gain` a `max_correction` určovali silu korekcie, ktorých hodnotu som tiež ladil hlavne od sledovania jazdy, aby neboli zásahy malé ani príliš trhané.

Testovanie sledovania objektu

Sledovanie objektu bolo založené na HSV segmentácii a výbere kontúry podľa tvarových vlastností. Najprv som nastavoval prahy offline podľa debug obrazu, potom som testoval správanie počas jazdy.

Intervaly HSV som upravoval tak, aby maska pokrývala loptu, ale nezachytávala okolie. Najcitlivejšie boli parametre $V_{\min}$ a $S_{\min}$, ktoré sa museli meniť podľa osvetlenia. Konkrétne v mojej zatienenej izbe sa maska často rozpadala, takže bolo potrebné nájsť kompromis medzi presnosťou a flexibilitou.

Morfologické operácie som ladil podľa šumu masky. Tvarové parametre ako kruhovitosť, solidita a vyplnenie som najprv nastavil prísnejšie, aby som eliminoval falošné detekcie, a potom mierne uvoľňoval, aby systém zvládol aj čiastočne zakrytý objekt. Na záver som doladil `min_confidence`, ktorý pomáhal vybrať najlepšiu kontúru.

Limity platformy

Jedným z hlavných obmedzení platformy je absencia enkodérov na kolesách. Odometria sa preto odhadovala bez priamej spätnej väzby z kolies. Poloha x, y vznikala spracovaním odosielaného `cmd_vel` a orientácia `yaw` bola preberaná z IMU. Systém tak predpokladal ideálne vykonanie príkazov, čo však neplatilo vždy a bolo to viditeľné pri pohybe modelu v RVize aj fyzického robota.

Ďalším zdrojom nepresnosti je gyroskop. Vo wander režime sa tento problém prejavoval napríklad pri otočkách, kde uhol odhadoval integráciou uhlovej rýchlosti ω_z , ktorá je citlivá na chyby merania. V dôsledku toho sa robot niekedy otočil o trochu viac alebo menej, než bolo zamýšľané; pri dlhšom behu sa drift môže prejaviť aj pomalou zmenou odhadu natočenia aj v pokoji.

4.11.1 Výsledky testov

Testovanie ukázalo, že systém je schopný spoľahlivo fungovať vo všetkých hlavných režimoch. Prepínanie medzi manuálnym a autonómnym riadením prebiehalo plynulo, bez potreby reštartu uzlov, čo potvrdzuje správny návrh komunikačnej vrstvy a mechanizmu prepínania režimov.

Režim wander dokázal reagovať na prekážky a pokračovať v pohybe bez zásahu používateľa. V meraniach odchýlky jazdy vychádza najlepšie optical flow, no jeho kvalita je citlivá na osvetlenie a textúru prostredia. Z pohľadu prevádzkovej stability musím ako lepšiu variantu vyhodnotiť IMU korekciu, ktorá fungovala použiteľne aj pri rôznych svetelných podmienkach a rozmanitosti prostredia.

Sledovanie lopty fungovalo po doladení HSV prahov a geometrických podmienok pomerne stabilne, pričom problémy sa opäť objavovali pri zmenách svetelných podmienok alebo v tieni. To ale patrí medzi bežné obmedzenia jednoduchých farebných segmentačných metód.

4.11.2 Diskusia výsledkov a ďalší vývoj

Získané výsledky potvrdili, že ROS 2 architektúra je vhodná pre modulárny návrh robotického systému. Jednotlivé komponenty bolo možné samostatne testovať, upravovať a kombinovať, čo uľahčilo vývoj aj ladenie.

Zároveň sa ukázalo, že presnosť celého systému je ovplyvnená hardvérovými obmedzeniami. Absencia enkodérov spôsobuje nepresnú odometriu, drift IMU ovplyvňuje stabilitu smeru a kamera je citlivá na zmeny osvetlenia. Tieto faktory sa navzájom kombinujú a priamo ovplyvňujú kvalitu jazdy aj detekcie objektov.

Získané poznatky naznačujú, že ďalší vývoj by sa mohol zamerať na doplnenie enkodérov, ktoré by poskytli presnejšiu spätnú väzbu o pohybe robota. Zlepšenie by v tomto ohľade priniesla aj pokročilejšia fúzia senzorov.

V oblasti kamery a spracovania obrazu by som do budúcnosti nahradil jednoduchú farebnú segmentáciu modernejšími prístupmi založenými na strojovom učení.

4.11.3 Uzly a ich funkcie v systéme

Pre lepšiu prehľadnosť slúži tabuľka 4.1. Ide o zhrnutie uzlov, ktoré priamo zodpovedajú funkcionalite opísanej v tejto kapitole.

Tabuľka 4.1: Hlavné ROS 2 uzly používané v práci

Uzol (ROS meno)	Účel
LD19 (ldlidar_ros2)	Ovládač LiDARu, publikuje sken na <code>/scan</code> .
imu_serial_fusion_bridge	Most z Arduino IMU do ROS 2, publikuje <code>/imu</code> .
camera_node (camera_ros)	Kamerový vstup, publikuje obraz a <code>camera_info</code> .
webRTC_camera (webRTC_camera_node.py)	Prenáša obraz do webového rozhrania cez WebRTC.
motor_hat_node (waverover_motor)	Riadi motory robota podľa príkazov rýchlosti.
lidar_wander_node (lidar_wander.py)	Jednoduché autonómne blúdenie podľa LiDARu, výstup na <code>/cmd_vel</code> .
robot_mode_switch (robot_mode_switch.py)	Prepína režimy robota cez ROS 2 služby (manual/wander/nav).
teleop_twist_keyboard	Klávesnicové manuálne ovládanie robota.
optical_flow_node (optical_flow)	Vizuálna korekcia smeru jazdy podľa optického toku.

Pokračuje na nasledujúcej strane

Tabuľka 4.1: Hlavné ROS 2 uzly používané v práci (pokračovanie)

Uzol (ROS meno)	Účel
<code>cmd_vel_odom</code> (<code>cmd_vel_odom.py</code>)	Odhad odometrie z príkazov rýchlosti, publikuje <code>/odom</code> a TF.
<code>base_link_to_imu</code> (<code>static_transform_publisher</code>)	Statická transformácia medzi telom robota a IMU.
<code>slam_toolbox</code> (<code>async_slam_toolbox_node</code>)	Tvorba 2D mapy a lokalizácia v mape (SLAM).
<code>ekf_filter_node</code> (<code>ekf_node</code>)	Fúzia senzorových dát cez EKF (<code>robot_localization</code>).
<code>odom_to_base_link</code> (<code>static_transform_publisher</code>)	Núdzová statická transformácia <code>odom</code> → <code>base_link</code> .
<code>robot_state_publisher</code>	Publikuje TF strom robota z URDF/Xacro modelu.
<code>rviz2</code>	Vizualizácia dát, mapy a TF počas testovania.
<code>rosbridge_websocket</code> , <code>rosapi</code>	Prepojenie webového rozhrania s ROS 2 cez WebSocket API.
<code>simple_nav_node</code> (<code>simple_nav_node.py</code>)	Jednoduchá bodová navigácia na zadaný cieľ.
<code>ball_follower</code> (<code>action_node.py</code>)	Deteguje a sleduje farebný objekt, riadi robot pri sledovaní lopty.
<code>ball_follow_bt_runner</code> (<code>bt_runner.py</code>)	Vyššia logika úlohy sledovania lopty pomocou behaviorálneho stromu.

Kapitola 5

Záver

Cieľom práce bolo navrhnuť a implementovať riadiaci softvér pre mobilnú platformu Wave Rover s využitím ROS 2 na Raspberry Pi. Počas práce sa mi podarilo vytvoriť funkčný systém, ktorý prepája jednotlivé hardvérové aj softvérové časti do jedného celku a umožňuje jeho ďalšie rozširovanie. Na základe implementácie a praktického testovania možno konštatovať, že stanovené ciele práce sa podarilo splniť.

Výsledkom je vlastný balík **waverover**, v ktorom som postupne implementoval riadenie pohonu, spracovanie dát zo senzorov, kamerový subsystém aj rôzne režimy riadenia. Systém som testoval na fyzickom robote aj v simulácii, čo mi umožnilo lepšie pochopiť rozdiely medzi ideálnym modelom a reálnym správaním platformy.

Počas implementácie a testovania som si všimol, že najväčšou výzvou nebolo samotné naprogramovanie jednotlivých častí, ale ich vzájomné prepojenie a ladenie. Mnohé problémy sa prejavili až pri jazde robota, napríklad nestabilita smeru alebo nepresnosti v odometrii, čo viedlo k postupným úpravám parametrov a celkového návrhu.

Praktické testovanie zároveň ukázalo aj limity použitej platformy. Absencia enkodérov sa prejavuje v nepresnej odometrii, IMU bez magnetometra vykazuje drift a optický tok je citlivý na osvetlenie a štruktúru prostredia. Tieto obmedzenia sú však typické pre jednoduchšie riešenia a bolo zaujímavé sledovať, ako sa navzájom ovplyvňujú.

Aj napriek objaveným nedostatkom je s touto platformou možné vytvoriť funkčný a modulárny robotický systém, ktorý je vhodný na experimentovanie a overovanie rôznych prístupov v prostredí ROS 2. Práca mi zároveň poskytla praktickú skúsenosť s návrhom a ladením robotického systému v reálnych podmienkach.

Do budúcnosti by sa projekt dal rozšíriť napríklad o enkodéry pre presnejšiu odometriu, lepšiu fúziu senzorových dát alebo pokročilejšie metódy spracovania obrazu, napríklad detekčné modely YOLO¹. Navrhnutý systém tak predstavuje dobrý základ pre ďalšie experimentovanie a rozvoj v oblasti mobilnej robotiky.

¹<https://arxiv.org/abs/1506.02640>

Literatúra

- [1] NAVE, C. R. *DC Motors* online. HyperPhysics, Georgia State University, 2000. Dostupné z: <https://hyperphysics.phy-astr.gsu.edu/hbase/magnetic/motdc.html>. [cit. 2025-10-04]. Stručný prehľad princípu jednosmerného motora.
- [2] NXP SEMICONDUCTORS. *PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller* datasheet. 2021. Dostupné z: <https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf>. [cit. 2026-04-30]. Datasheet čipu PCA9685 použitého na generovanie PWM pre motorový HAT.
- [3] ANALOG DEVICES. *Accelerometer and Gyroscopes Sensors: Operation, Sensing, and Applications* online. 2024. Dostupné z: <https://www.analog.com/en/resources/technical-articles/accelerometer-and-gyroscopes-sensors-operation-sensing-and-applications.html>. [cit. 2025-10-11]. Prehľad princípov MEMS akcelerometra a gyroskopu, aplikácie a súvislosti s fúziou dát.
- [4] MAHONY, R.; HAMEL, T. a PFLIMLIN, J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group. *IEEE Transactions on Automatic Control*, 2010, zv. 55, č. 5, s. 1203–1218. ISSN 0018-9286. Dostupné z: <https://hal.science/hal-00488376/document>. Teoretický rámec komplementárnej filtrácie orientácie na $SO(3)$; úplný text voľne na HAL.
- [5] MAYITZÍN, A. *AHRS: Complementary filter* Dokumentácia balíka AHRS. 2025. Dostupné z: <https://ahrs.readthedocs.io/en/latest/filters/complementary.html>. [cit. 2025-10-13]. Praktický popis diskkrétnej zmiešanej formy odhadu z integrácie gyroskopu a akcelerometra s koeficientom alfa (vektorová forma zahŕňa jednoosový prípad).
- [6] ENCYCLOPAEDIA BRITANNICA. *Lidar* online. Encyclopaedia Britannica, 2026. Dostupné z: <https://www.britannica.com/technology/lidar>. [cit. 2025-10-18]. Všeobecný popis princípu LiDAR a oblastí použitia.
- [7] NXP SEMICONDUCTORS. *I2C-bus specification and user manual* User manual UM10204. 2021. Dostupné z: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>. [cit. 2025-10-28]. Špecifikácia a používateľská príručka zbernice I2C.
- [8] ANALOG DEVICES. *UART: A Hardware Communication Protocol* online. 2026. Dostupné z: <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>. [cit. 2026-04-30]. Technický prehľad asynchrónnej UART komunikácie: rámec, baud rate, linky TX/RX.

- [9] OPEN ROBOTICS. *ROS 2 Documentation* online. 2025. Dostupné z: <https://docs.ros.org/en/jazzy>. [cit. 2025-11-01]. Distribúcia Jazzy; tutoriály a koncepty.
- [10] OBJECT MANAGEMENT GROUP / DDS FOUNDATION. *What is DDS?* online. 2024. Dostupné z: <https://www.dds-foundation.org/what-is-dds-3/>. [cit. 2025-12-01]. Štandard middleware pod ROS 2.
- [11] GAZEBO SIM DOCUMENTATION. *ROS 2 Integration* online. 2024. Dostupné z: https://gazebo-sim.org/docs/garden/ros2_integration/. [cit. 2026-04-30]. Integrácia Gazebo a ROS 2, vrátane použitia `ros_gz_bridge`.
- [12] ONDERKA, D. *Mobilní kamera realizovaná prostředky ROS2*. 2024. Bakalárska práca. Vysoké učení technické v Brně, Fakulta informačních technologií. Dostupné z: <http://hdl.handle.net/11012/247834>. [cit. 2026-01-01]. Práca dostupná v DSpace VUT; inšpirácia štruktúrou implementačnej kapitoly.
- [13] CORKE, P. a HAVILAND, J. *Robotics, Vision and Control: Differential Drive Robot* online. 2023. Dostupné z: https://www.roboticsbook.org/S52_diffdrive_actions.html. [cit. 2026-04-30]. Odvodenie kinematiky diferenciálneho podvozku a vzťah medzi lineárnou a uhlovou rýchlosťou.
- [14] ÅSTRÖM, K. J. a MURRAY, R. M. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. ISBN 978-0-691-13576-2. Dostupné z: https://www.cds.caltech.edu/~murray/books/AM08/pdf/fbs-pid_24Jul2020.pdf. Učebnica spätnej väzby a regulácie; vhodný zdroj k PID regulátoru.
- [15] LUCAS, B. D. a KANADE, T. An Iterative Image Registration Technique with an Application to Stereo Vision. In: *Proceedings of the 7th International Joint Conference on Artificial Intelligence*. 1981, s. 674–679. ISBN 978-0-86576-059-2. Dostupné z: <https://idl.uw.edu/living-papers-paper/lucas-kanade/>. Pôvodná práca Lucas–Kanade metódy použitej pri optickom toku. ISBN zborníka IJCAI-81 (William Kaufmann).
- [16] SHI, J. a TOMASI, C. Good Features to Track. In: *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*. 1994, s. 593–600. ISBN 978-0-8186-5825-9. Dostupné z: https://publications.ri.cmu.edu/storage/publications/pub_files/pub2/shi_jianbo_1994_1/shi_jianbo_1994_1.pdf. Pôvodná práca ku výberu sledovateľných bodov (Shi–Tomasi). ISBN papierového zborníka CVPR 1994 (IEEE Computer Society).
- [17] OPENCV. *Thresholding Operations using inRange* online. 2025. Dostupné z: https://docs.opencv.org/3.4/da/d97/tutorial_threshold_inRange.html. [cit. 2026-03-18]. Oficiálny tutoriál OpenCV pre farebnú segmentáciu v priestore HSV.
- [18] OPENCV. *Morphological Operations* online. 2025. Dostupné z: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html. [cit. 2026-04-30]. Oficiálny tutoriál OpenCV k morfológickým operáciám (opening, closing).

- [19] OPENCV. *Contours: Getting Started* online. 2025. Dostupné z: https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html. [cit. 2026-04-30]. Oficiálny tutoriál OpenCV k nájdeniu a práci s kontúrami (`findContours`).
- [20] PY_TREES DEVELOPERS. *Py_trees documentation* online. 2026. Dostupné z: <https://py-trees.readthedocs.io/>. [cit. 2026-04-30]. Dokumentácia knižnice `py_trees` pre behaviorálne stromy.
- [21] ROBOT WEB TOOLS. *Roslibjs* online. 2024. Dostupné z: <https://github.com/RobotWebTools/roslibjs>. [cit. 2026-04-13]. JavaScript klientská knižnica pre ROS cez `rosbridge`.
- [22] ROBOT WEB TOOLS. *Rosbridge_suite* online. 2024. Dostupné z: https://github.com/RobotWebTools/rosbridge_suite. [cit. 2026-04-04]. WebSocket most medzi prehliadačom a ROS 2 (`rosbridge`).
- [23] KINGSTON TECHNOLOGY. *Industrial Grade microSD Cards* online. 2026. Dostupné z: <https://www.kingston.com/en/memory-cards/industrial-grade-microsd-uhs-i-u3>. [cit. 2026-04-30]. Priemyselné microSD karty určené pre náročné podmienky a vyššiu zápisovú záťaž.

Príloha A

Plagát

Plagát k bakalárskej práci je zobrazený na obr. A.1.



Obr. A.1: Plagát k bakalárskej práci